



UNIVERSITÄT
LEIPZIG

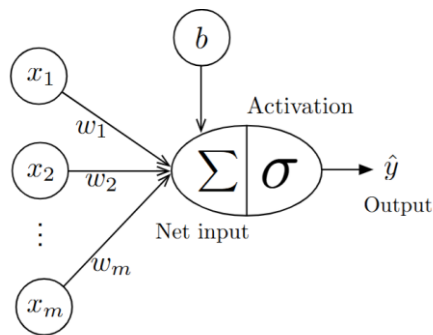
Transformers 2: The Transformer Architecture

Natural Language Processing

Leonie Weissweiler

RECAP

THE PERCEPTRON UPDATE RULE



$$\sigma \left(\sum_{i=1}^m x_i w_i + b \right) = \sigma (\mathbf{x}^T \mathbf{w} + b) = \hat{y} \quad \sigma(z) = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

Inputs

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$

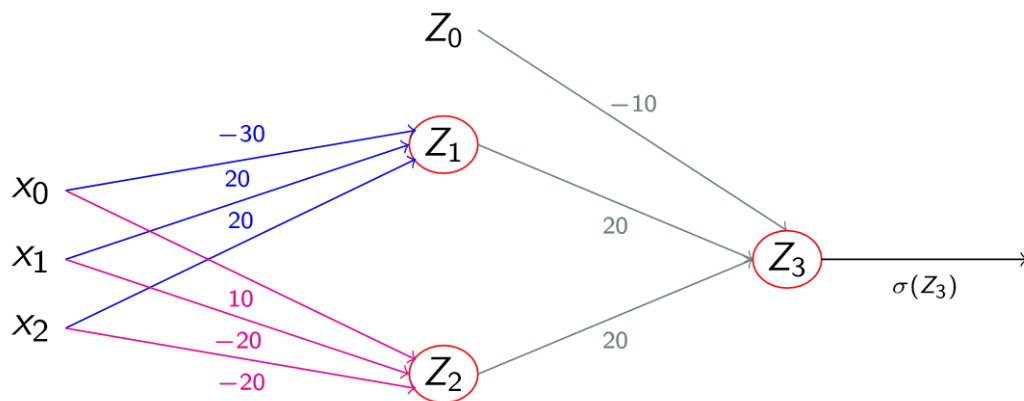
Initialize $\mathbf{w} := \mathbf{0}^{m-1}$, $b := 0$

For every training epoch:

For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:

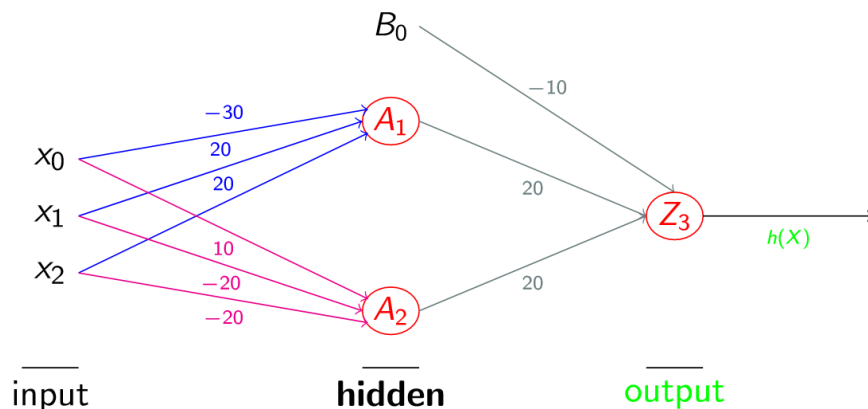
- (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$ ← Compute output (prediction)
- (b) $err := (y^{[i]} - \hat{y}^{[i]})$ ← Calculate error
- (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$, $b := b + err$ ← Update parameters

FUNCTION COMPOSITION



x_0	x_1	x_2	$\sigma(Z_1)$	$\sigma(Z_2)$	$\sigma(Z_3)$
1	0	0	≈ 0	≈ 1	$\sigma(1 * -10 + 0 * 20 + 1 * 20) = \sigma(10) \approx 1$
1	0	1	≈ 0	≈ 0	$\sigma(1 * -10 + 0 * 20 + 0 * 20) = \sigma(-10) \approx 0$
1	1	0	≈ 0	≈ 0	$\sigma(1 * -10 + 0 * 20 + 0 * 20) = \sigma(-10) \approx 0$
1	1	1	≈ 1	≈ 0	$\sigma(1 * -10 + 1 * 20 + 1 * 20) = \sigma(30) \approx 1$

FEEDFORWARD NEURAL NETWORK



Computation of hidden layer $\mathbf{H}$

$$A_1 = \sigma(X \cdot \Theta_1)$$

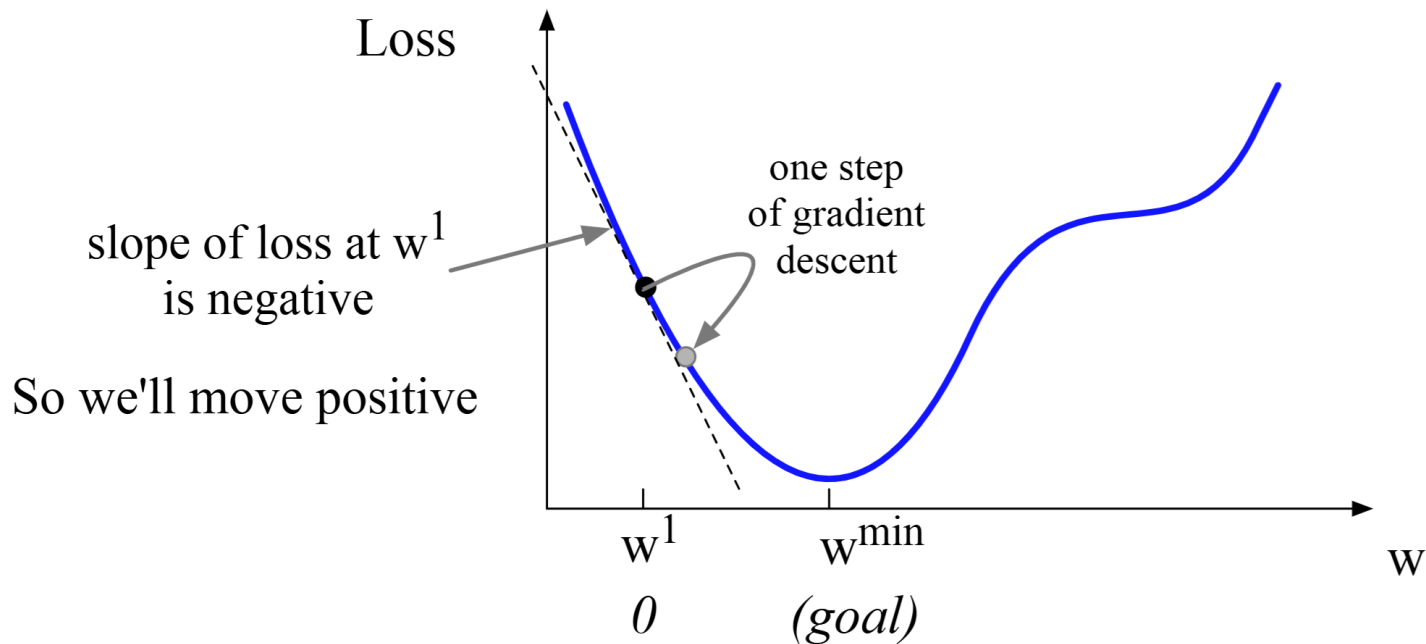
$$A_2 = \sigma(X \cdot \Theta_2)$$

$$B_0 = 1 \text{ (bias term)}$$

Computation of output unit

$$h(X) = \sigma(\mathbf{H} \cdot \Theta_3)$$

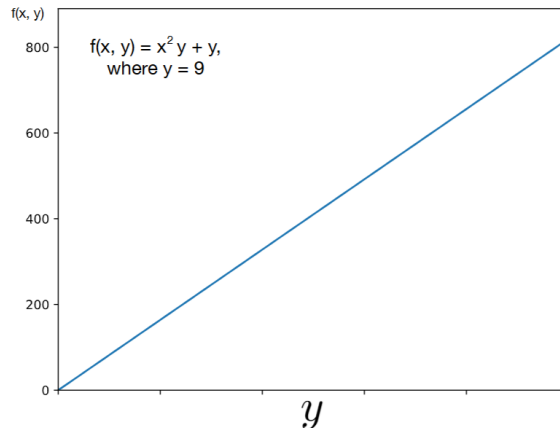
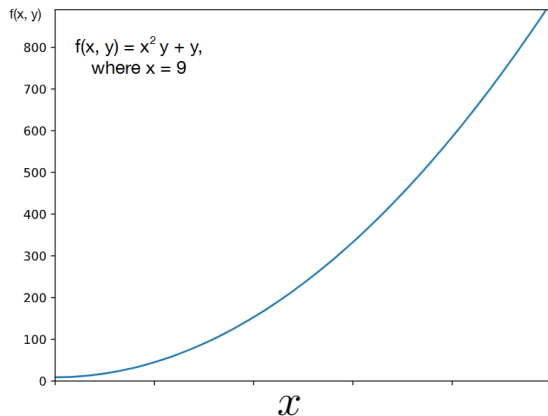
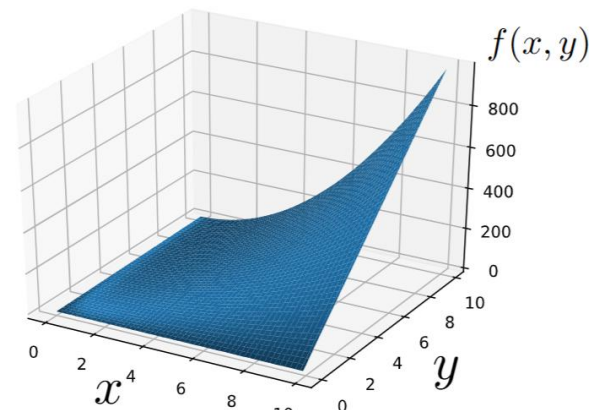
OUR GOAL: MINIMISING THE LOSS



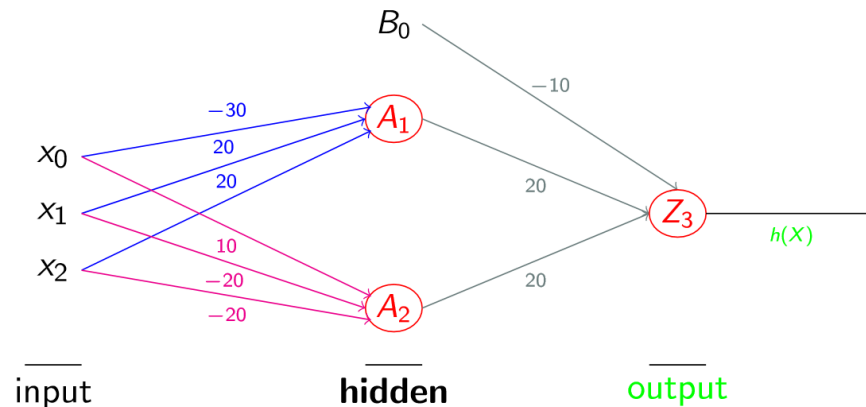
GRADIENTS OF MULTIPLE VARIABLES AT ONCE

$$f(x, y) = x^2 y + y$$

$$\nabla f(x, y) = \begin{bmatrix} 2xy \\ x^2 + 1 \end{bmatrix}$$



BACKPROPAGATION

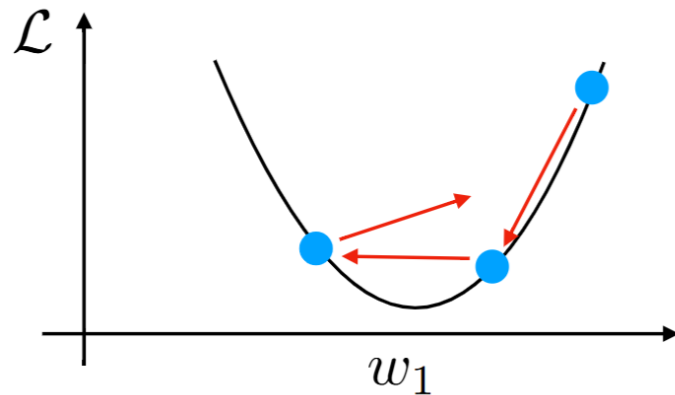


Intuition

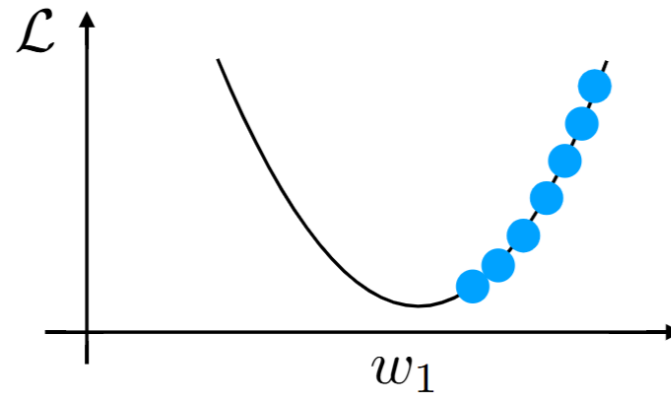
- One derivative going in the opposite direction of each arrow
- Here, adjusting A_1 and A_2 are simple derivatives
- Adjusting x_0 , x_1 and x_2 requires the chain rule, but we can re-use the derivatives for A_1 and A_2

LEARNING RATE

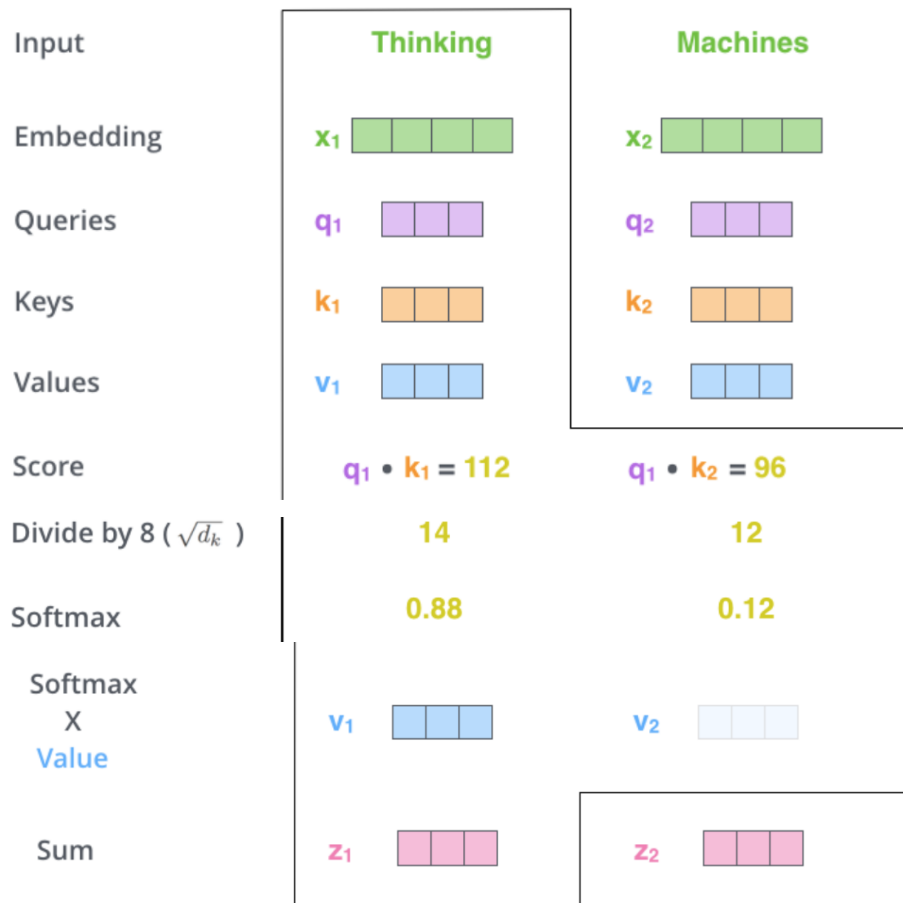
Learning rate too large $\rightarrow$ overshoot



learning rate too small $\rightarrow$ slow convergence



Transformer Architecture



Vector multiplication

Divide by square root of dimensionality
→ more stable gradients

Softmax → drown out irrelevant values of v by multiplying them with tiny numbers

MATRIX CALCULATION OF SELF-ATTENTION

$$\begin{array}{c}
 \text{X} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \text{W}^{\text{Q}} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 \text{Q} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}$$

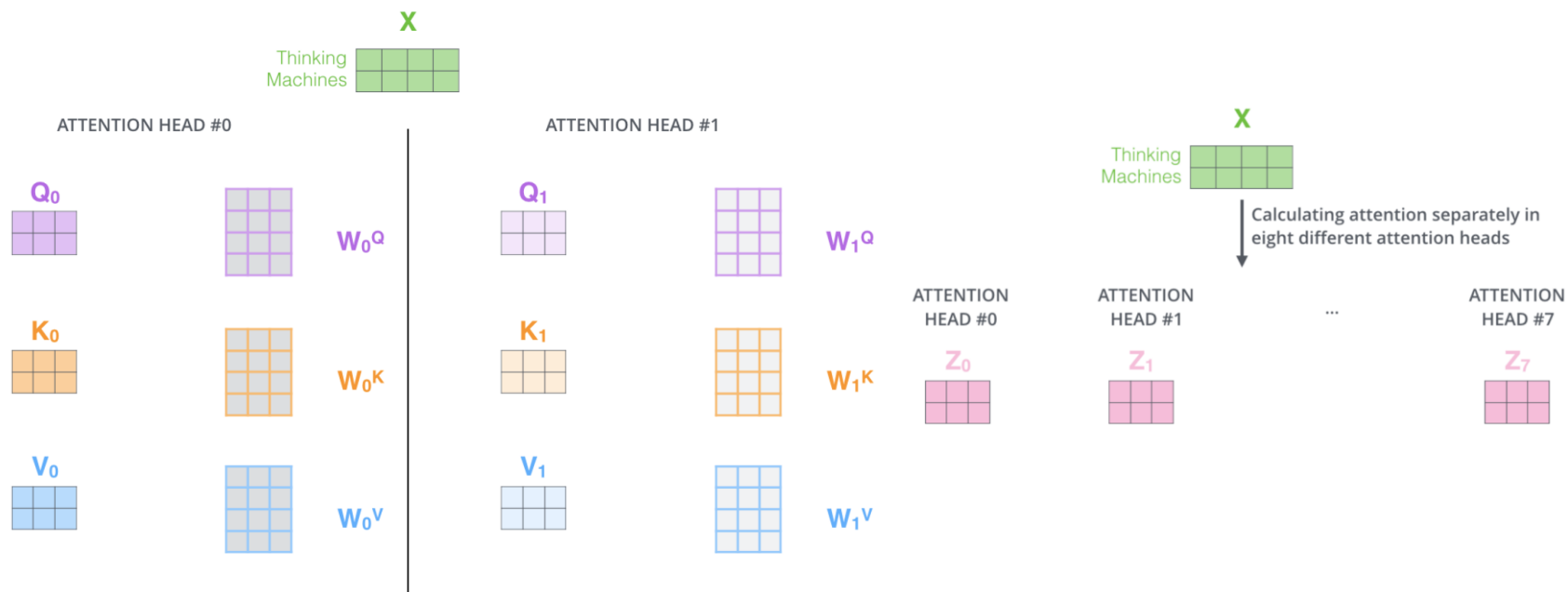
$$\begin{array}{c}
 \text{X} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \text{W}^{\text{K}} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 \text{K} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}$$

$$\begin{array}{c}
 \text{X} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \text{W}^{\text{V}} \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 \text{V} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}$$

$$\text{softmax} \left(\frac{
 \begin{array}{c}
 \text{Q} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \text{K}^{\text{T}} \\
 \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}
 \end{array}
 }{
 \sqrt{d_k}
 } \right)
 \begin{array}{c}
 \text{V} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}$$

$$=
 \begin{array}{c}
 \text{Z} \\
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}
 \end{array}$$

MULTI-HEAD ATTENTION



Transformer Architecture

1) This is our input sentence*

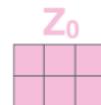
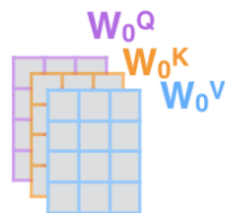
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

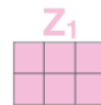
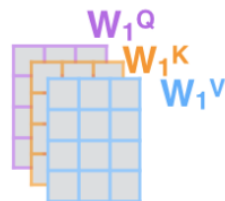
Thinking
Machines



W^O



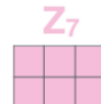
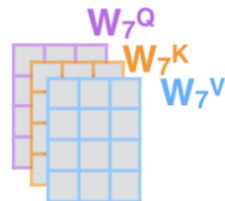
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

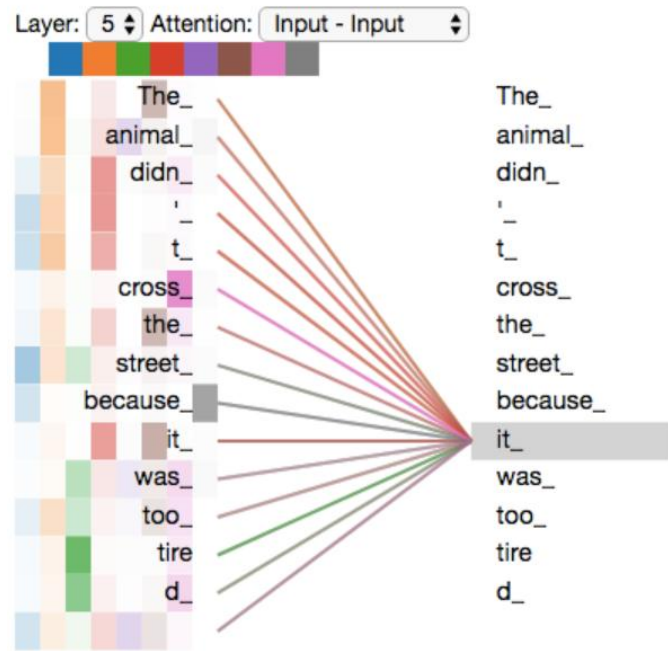
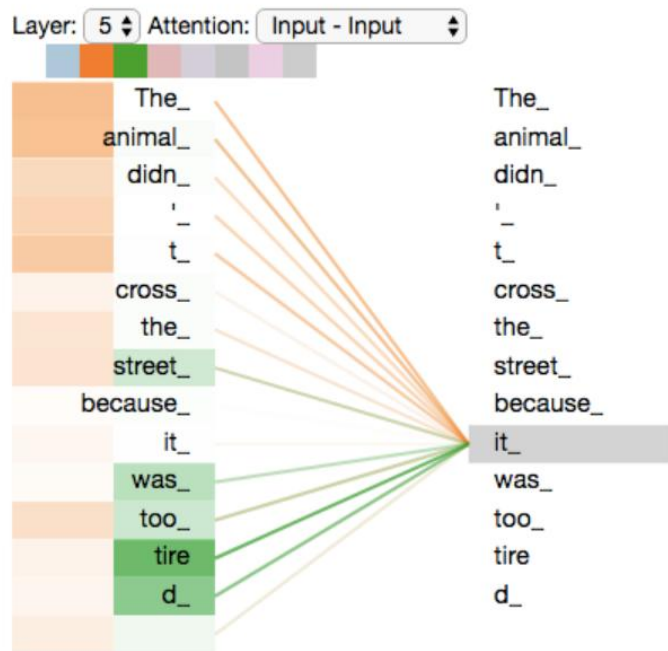


...

...

...





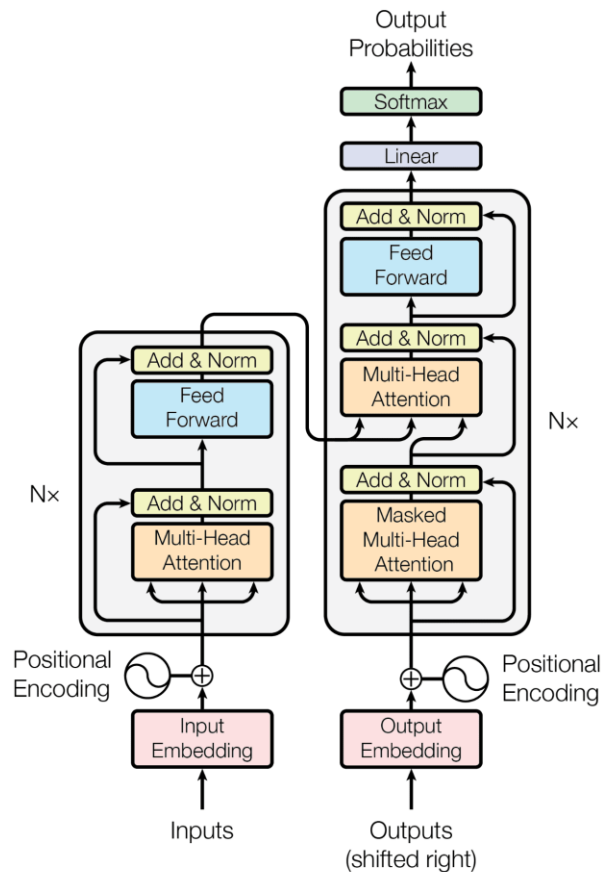
QUESTIONS?

TRANSFORMERS (FOR REAL THIS TIME)

OUTLINE OF TODAY

- The big scary graph
- Encoder-Decoder models
- Attention again, but this time with encoder-decoder attention
- Multi-head attention again
- Other components for transformers
- Putting it all together
- The big not-so-scary graph

THE BIG SCARY GRAPH

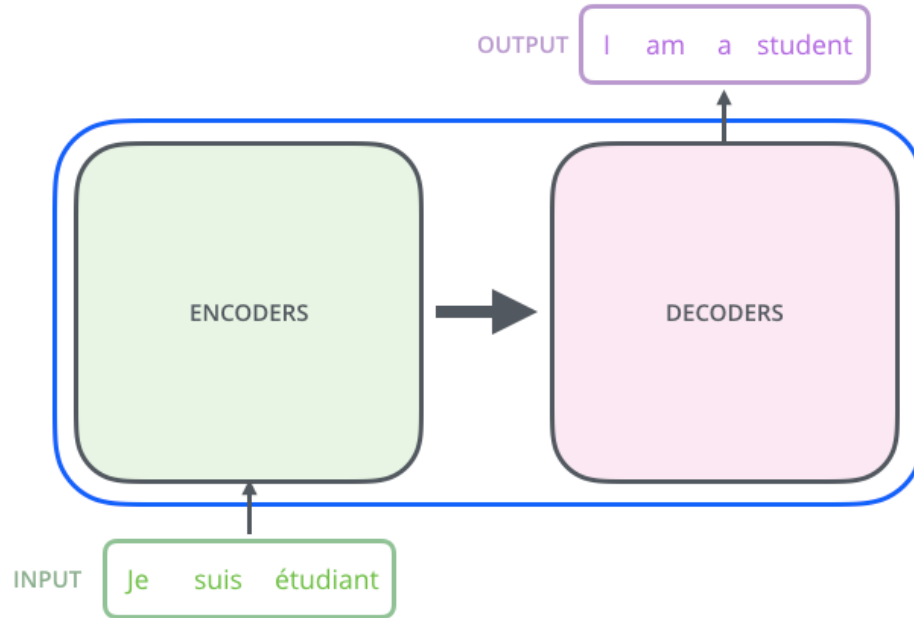


ENCODER-DECODER ARCHITECTURES

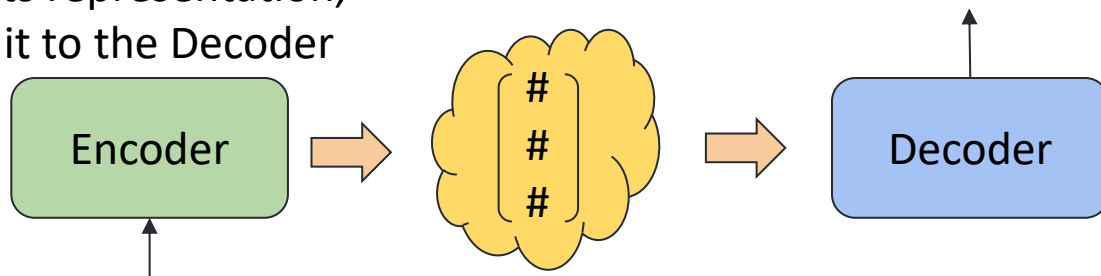
MACHINE TRANSLATION



ENCODER-DECODER MODELS



Encoder reads the *source*,
produces its representation,
and gives it to the Decoder



Decoder uses the
source representation
to generate the *output*

CONDITIONAL LANGUAGE MODELS

Language Models:

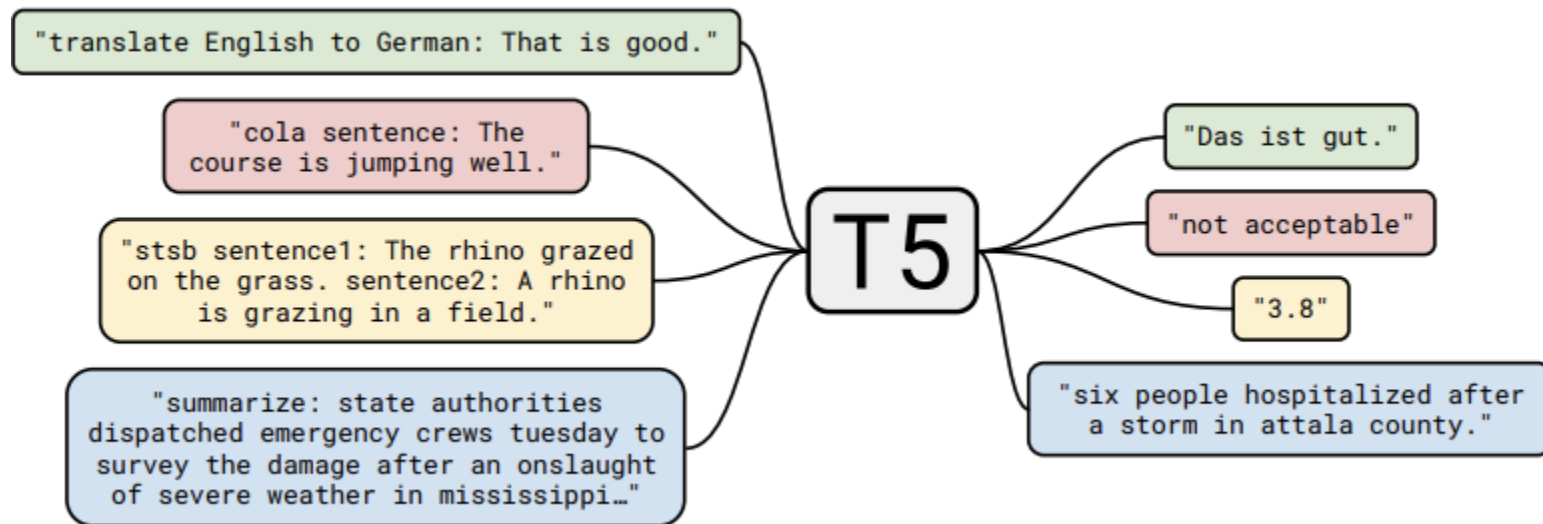
$$P(y_1, y_2, \dots, y_n) = \prod_{t=1}^n p(y_t | y_{<t})$$

Conditional Language Models:

$$P(y_1, y_2, \dots, y_n, \underset{\substack{\uparrow \\ \text{condition on source } x}}{x}) = \prod_{t=1}^n p(y_t | y_{<t}, x)$$

$\mathcal{X}$ does not always need to be a sequence of tokens. (e.g. image captioning)

SEQUENCE-TO-SEQUENCE TASKS



MORE SEQUENCE-TO-SEQUENCE TASKS

Logical Inference

Input: Shelley is from Virginia, but is visiting that city with that famous market where they throw the fish. Going home next Tuesday!

Question: Is it likely that Shelley will be near the Pacific Ocean this weekend?

Model Output: The city with the famous market where they throw the fish is Seattle, Washington. Seattle is on the Pacific Ocean. Shelley is visiting Seattle, so she will be near the Pacific Ocean this weekend. The answer is "yes", it is likely that Shelley will be near the Pacific Ocean this weekend.

ENCODER VS DECODER

Encoder

Who is doing:

- all source tokens

What they are doing:

- look at each other
 - update representations
- repeat
N times

Decoder

Who is doing:

- target token at the current step

What they are doing:

- looks at previous target tokens
 - looks at source representations
 - update representation
- repeat
N times

PARALLEL PROCESSING

I arrived at the **bank** after crossing thestreet? ...river?

What does **bank** mean in this sentence?



I've no idea: let's wait until I read the end

RNNs

$O(N)$ steps to process a sentence with length N

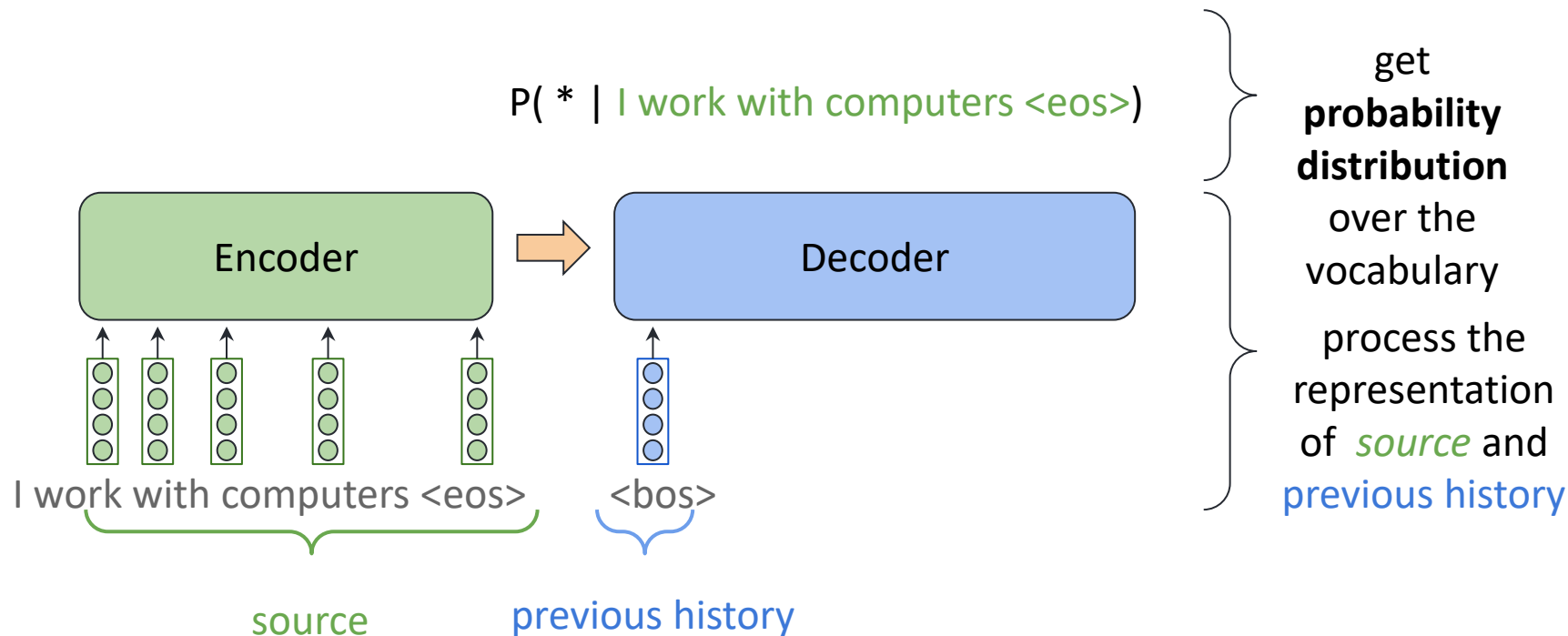


I don't need to wait - I see all words at once!

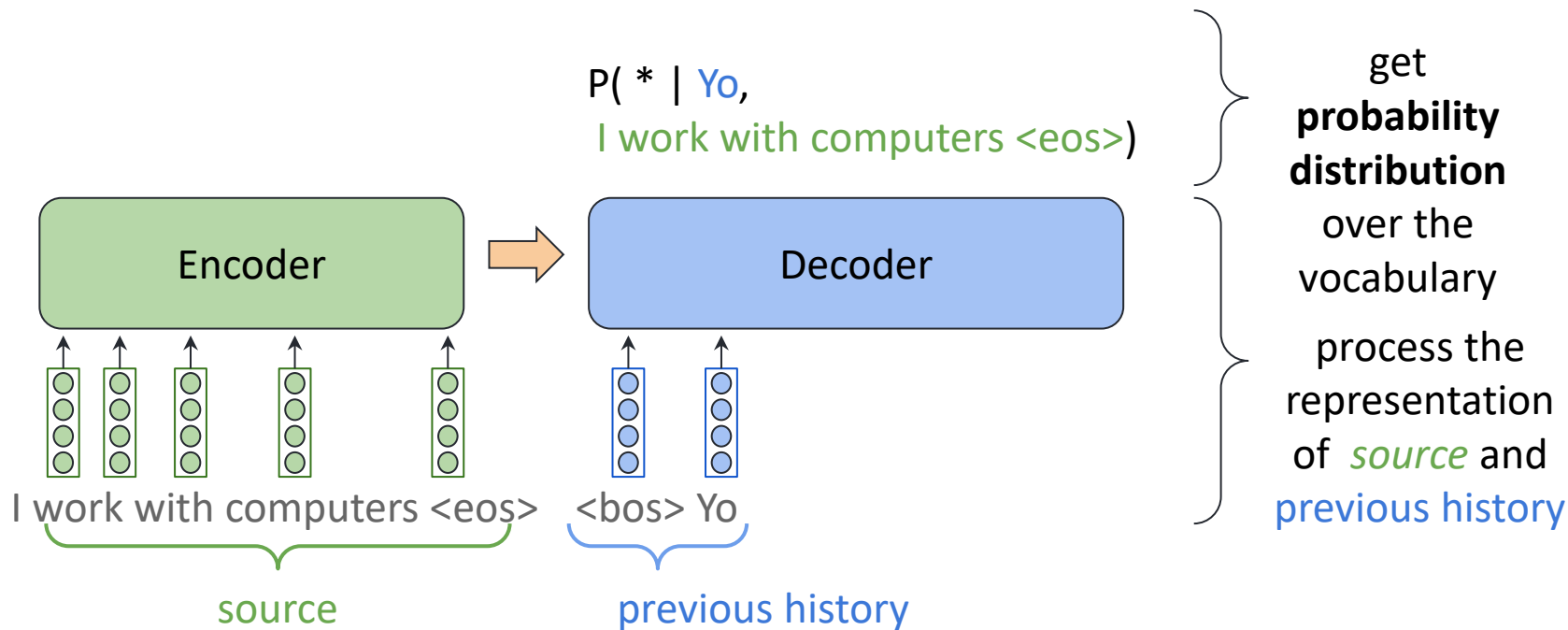
Transformer

Constant number of steps to process any sentence

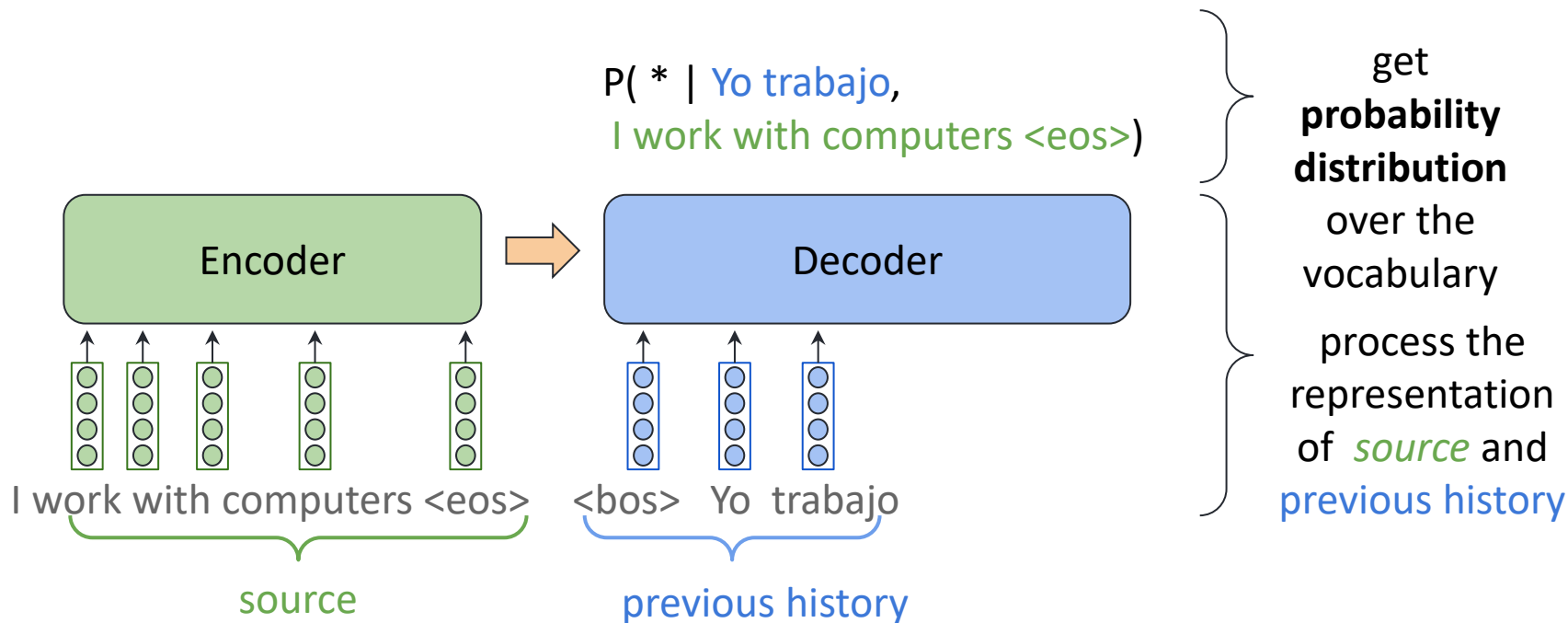
SEQUENCE GENERATION



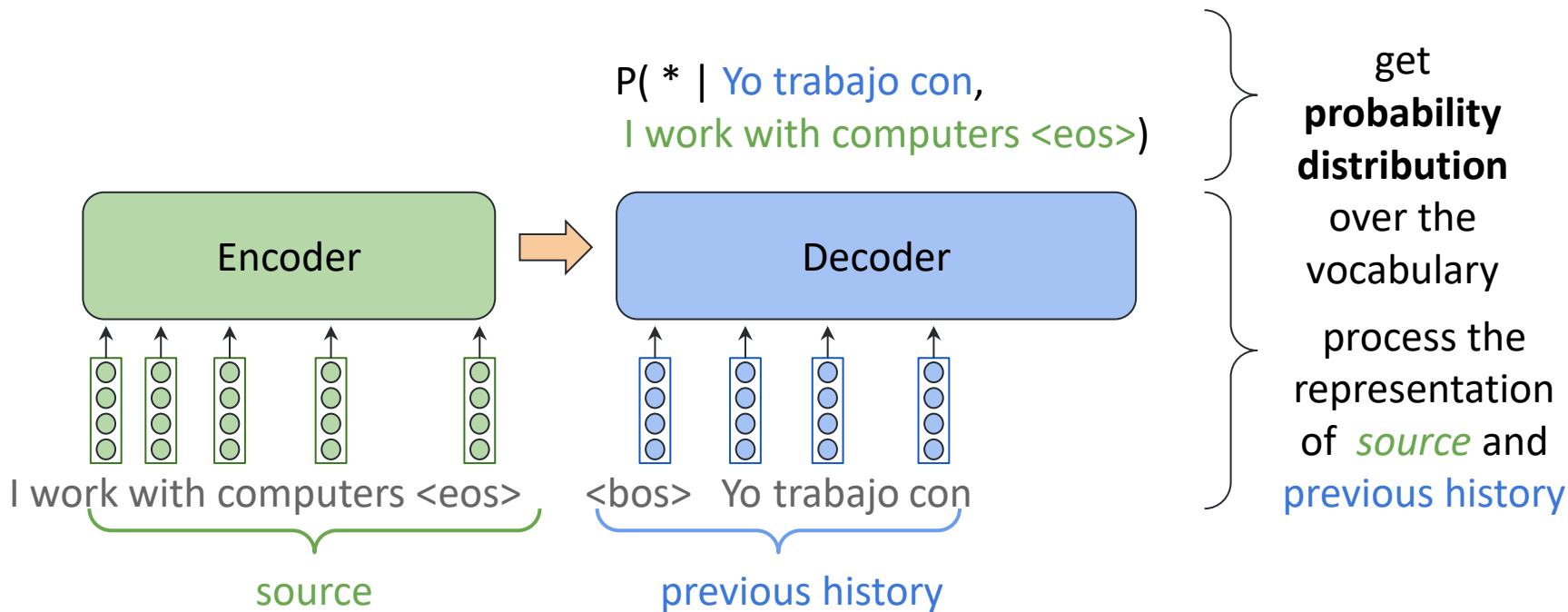
SEQUENCE GENERATION



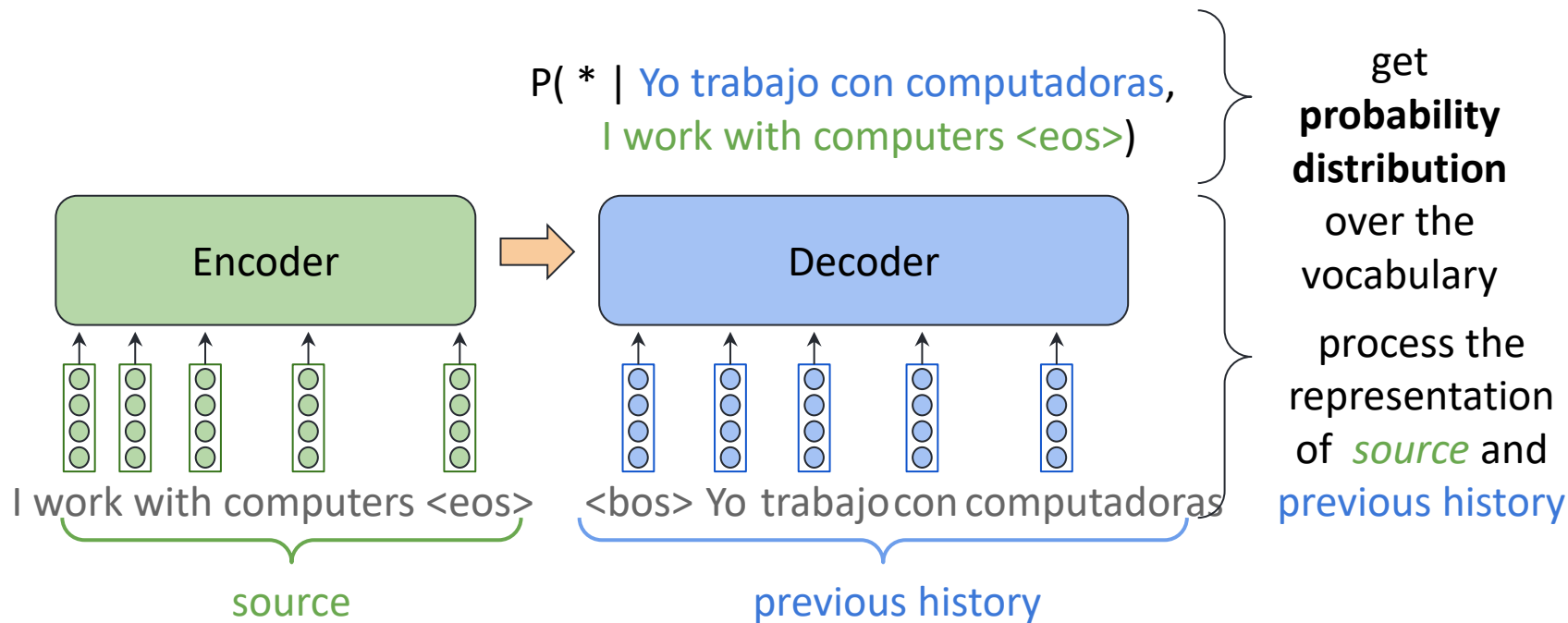
SEQUENCE GENERATION



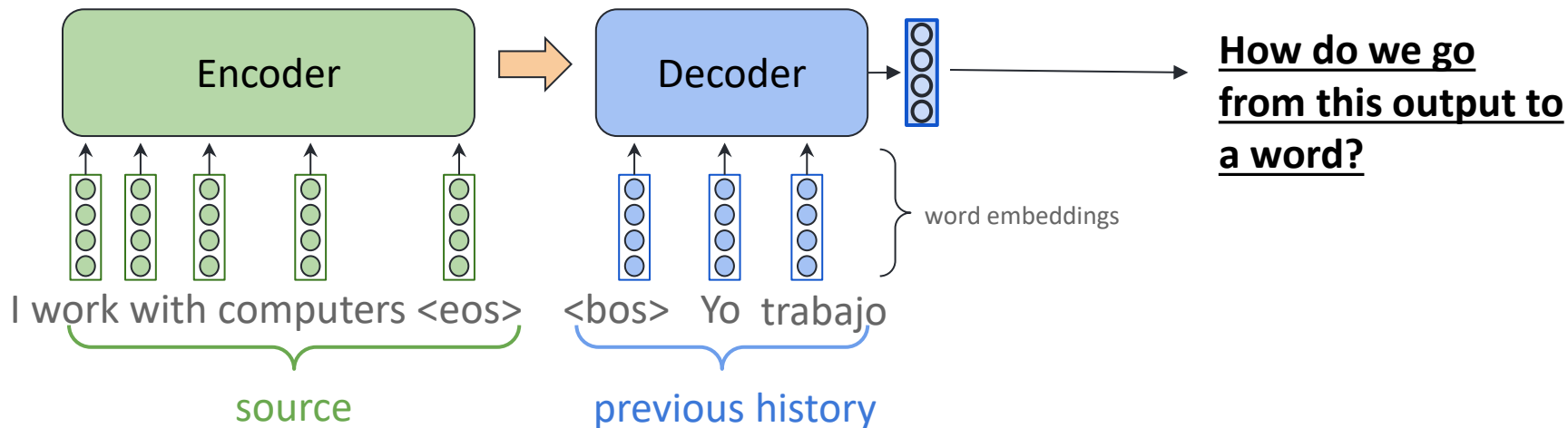
SEQUENCE GENERATION



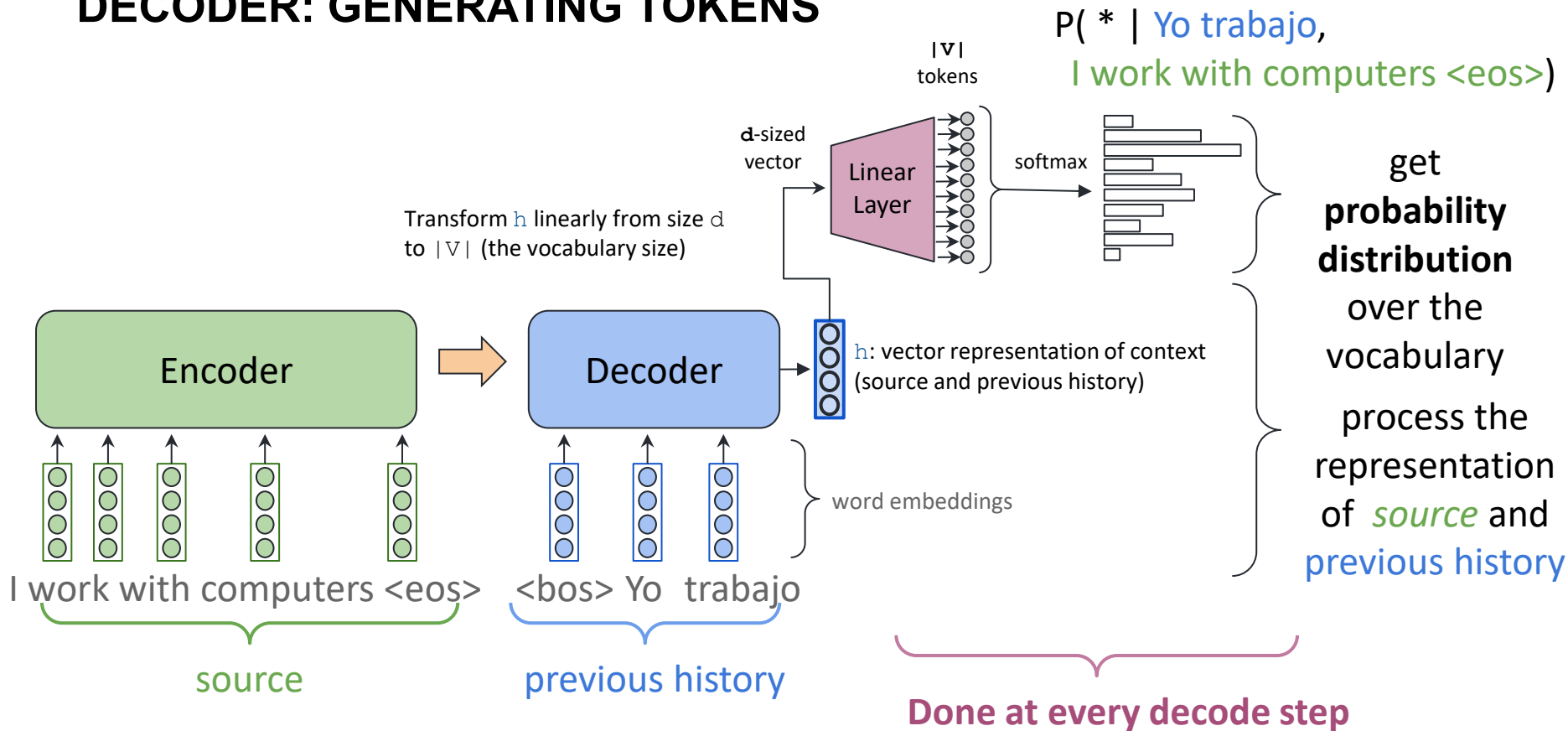
SEQUENCE GENERATION



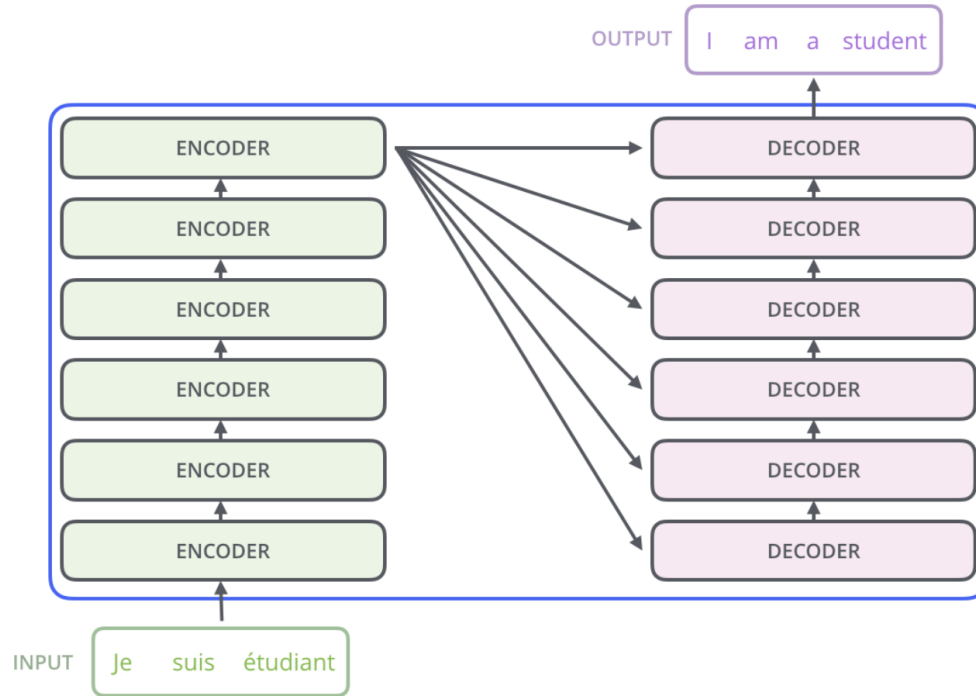
DECODER: GENERATING TOKENS



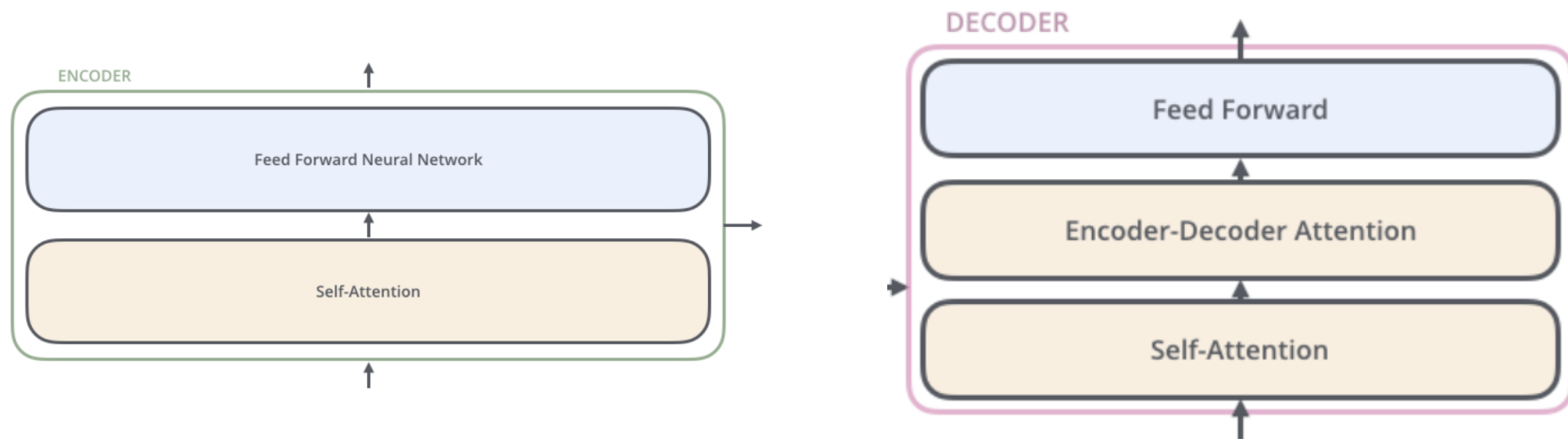
DECODER: GENERATING TOKENS

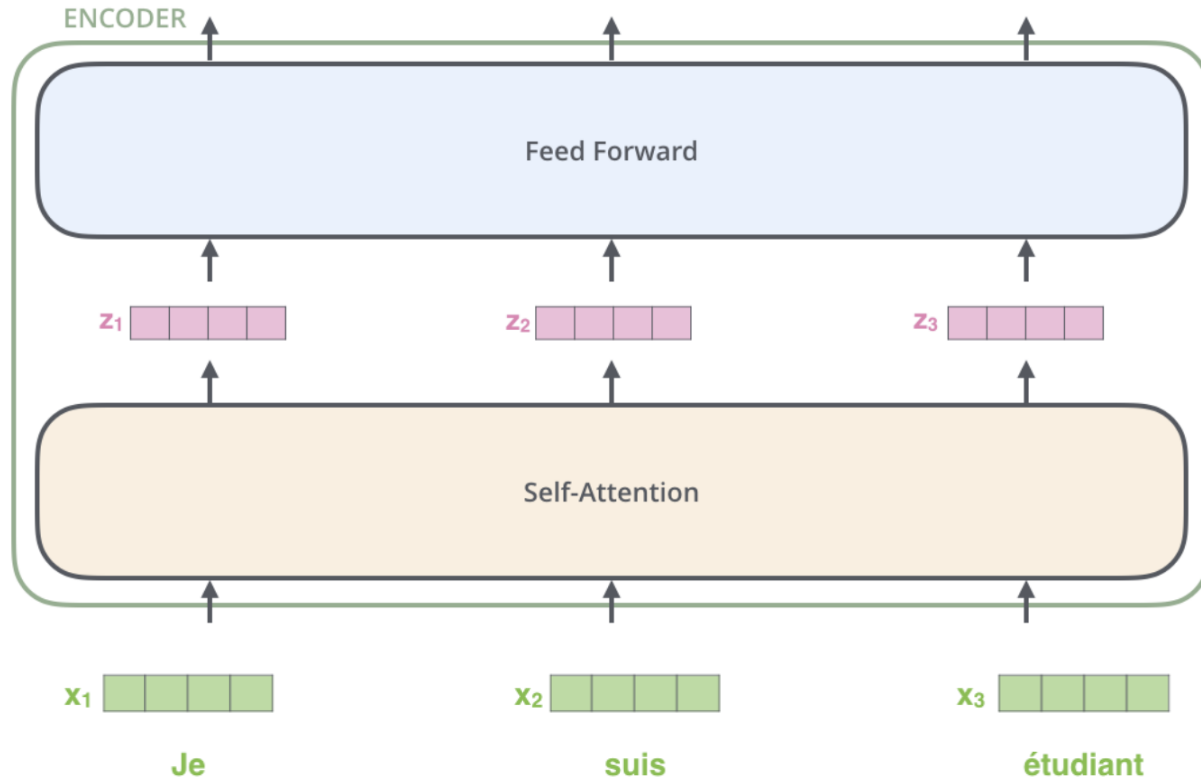


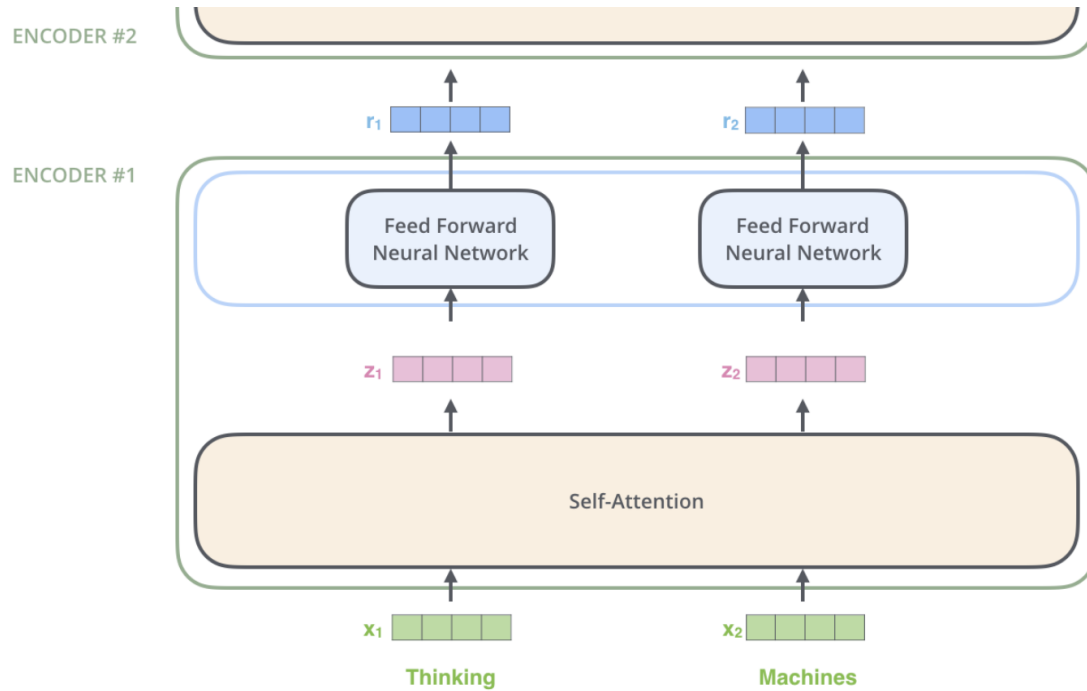
ENCODER AND DECODER IN BLOCKS



ENCODER BLOCKS AND DECODER BLOCKS







ATTENTION AGAIN – NOW NOT ONLY SELF-ATTENTION

SELF-ATTENTION

The self-attention for each word consists of a:

“Query” → Think of it as the question that tries to establish
this word’s context.

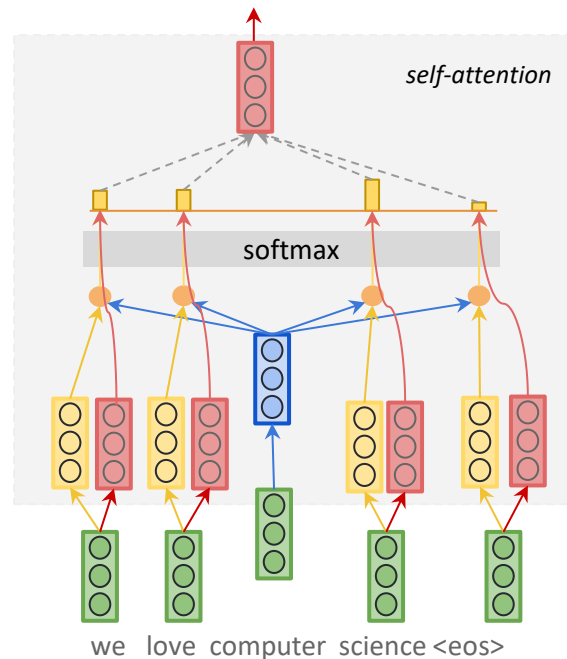
“Key” → (Query * Key) gives the answer to how good
this word is at providing context to the querying
word (notice, this can be a different word)

“value” → The value of each word

SELF-ATTENTION: QUERY, KEY, VALUE

Each vector receives a:

- ⚡ **Query (q):** vector **from** which the attention is looking
- ⚡ **Key (k):** vector **at** which the query looks to establish context
- ⚡ **Value (v):** value of word being looked **at**, weighted based on context

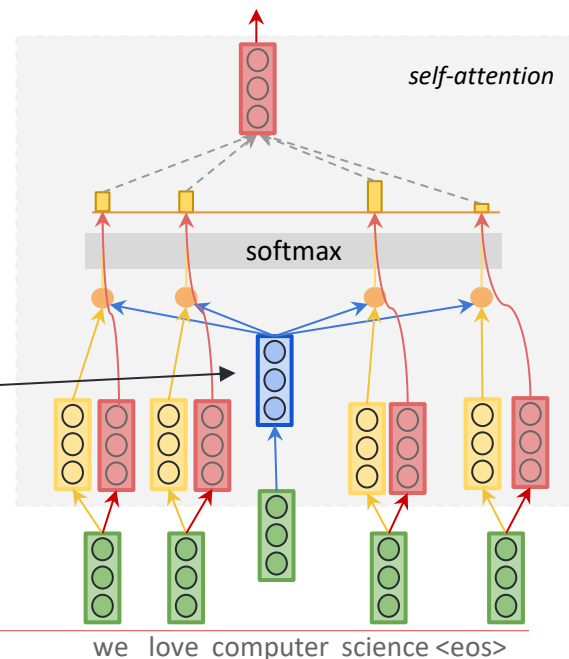


SELF-ATTENTION: QUERY, KEY, VALUE

Each vector receives a:

- ⌘ **Query (q)**: vector **from** which the attention is looking
- ⌘ **Key (k)**: vector **at** which the query looks to establish context
- ⌘ **Value (v)**: value of word being looked **at**, weighted based on context

The **Query** aims to establish the context, in this example, of the word “computer” (**from**)



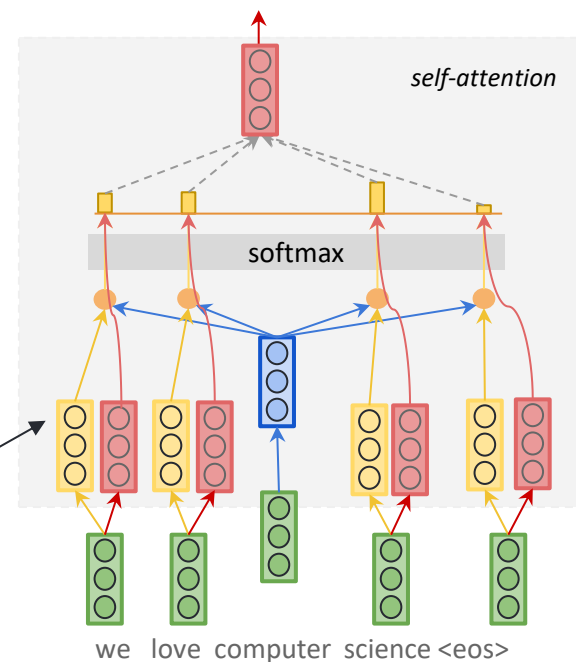
SELF-ATTENTION: QUERY, KEY, VALUE

Each vector receives a:

- ⚡ **Query (q):** vector **from** which the attention is looking
- ⚡ **Key (k):** vector **at** which the query looks to establish context
- ⚡ **Value (v):** value of word being looked **at**, weighted based on context

The **Query** aims to establish the context, in this example, of the word “computer” (**from**)

Done by querying each **Key** of all words (**to**)

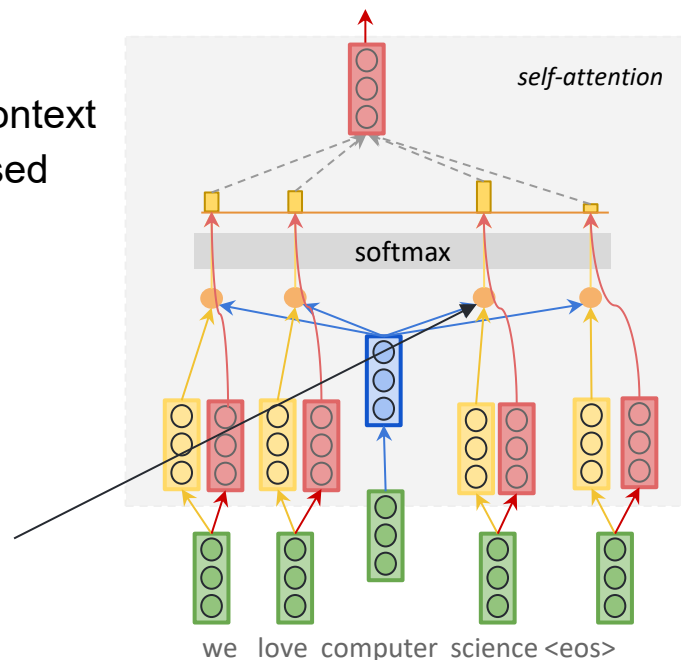


SELF-ATTENTION: QUERY, KEY, VALUE

Each vector receives a:

- ⚡ **Query (q)**: vector **from** which the attention is looking
- ⚡ **Key (k)**: vector **at** which the query looks to establish context
- ⚡ **Value (v)**: value of word being looked **at**, weighted based on context

In our example, this might establish “science” as the word the provides the most informative context.

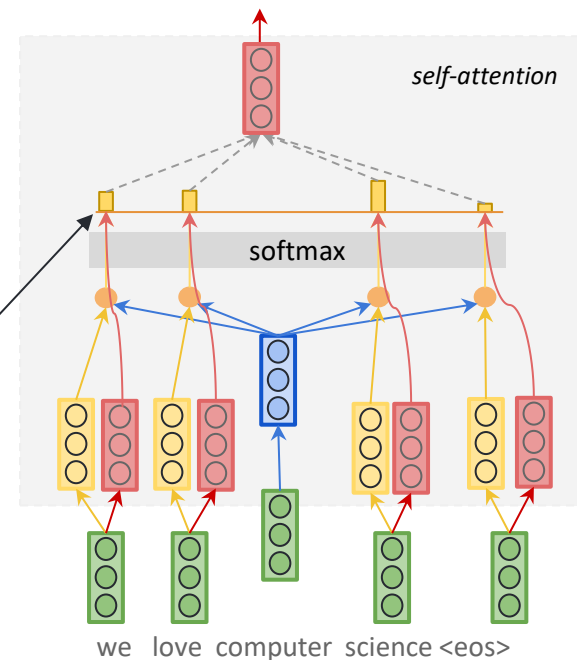


SELF-ATTENTION: QUERY, KEY, VALUE

Each vector receives a:

- € **Query (q)**: vector **from** which the attention is looking
- € **Key (k)**: vector **at** which the query looks to establish context
- € **Value (v)**: value of word being looked **at**, weighted based on context

We push the output of this through a softmax and multiply each with the **values** associated with each word (target word, i.e. “at”)



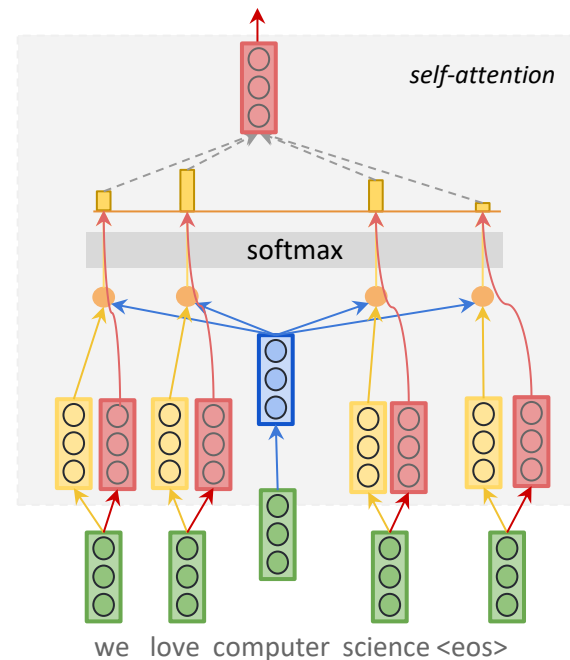
SELF-ATTENTION: QUERY, KEY, VALUE 💡

Each vector receives a:

- ⚡ **Query (q):** vector **from** which the attention is looking
- ⚡ **Key (k):** vector **at** which the query looks to establish context
- ⚡ **Value (v):** value of word being looked **at**, weighted based on context

$$\text{Attention}(\underset{\substack{\text{from}}}{q}, \underset{\substack{\text{to}}}{k}, v) = \text{softmax} \left(\overbrace{\frac{qk^T}{\sqrt{d_k}}}^{\text{Attention Weights}} \right) v$$

Vector dimensionality of K, V



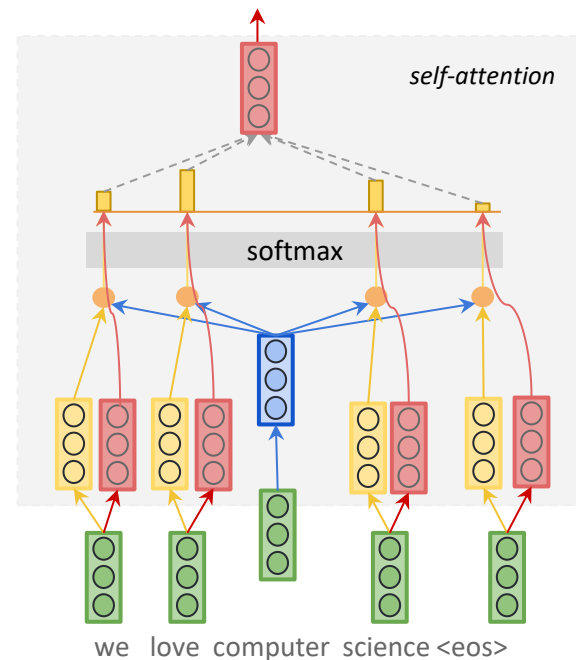
SELF-ATTENTION: QUERY, KEY, VALUE 💡

Each vector receives a:

- ⚡ **Query (q):** vector **from** which the attention is looking
- ⚡ **Key (k):** vector **at** which the query looks to establish context
- ⚡ **Value (v):** value of word being looked **at**, weighted based on context

$$\text{Attention}(\underset{\substack{\text{from}}}{q}, \underset{\substack{\text{to}}}{k}, v) = \text{softmax} \left(\overbrace{\frac{qk^T}{\sqrt{d_k}}}^{\text{Attention Weights}} \right) v$$

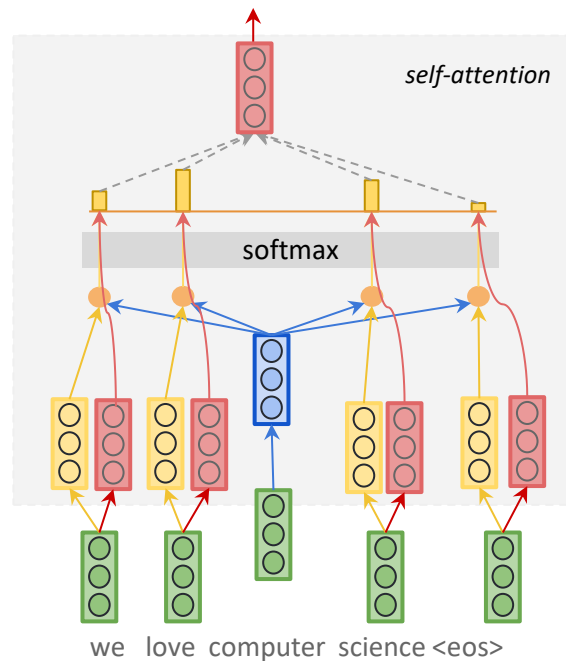
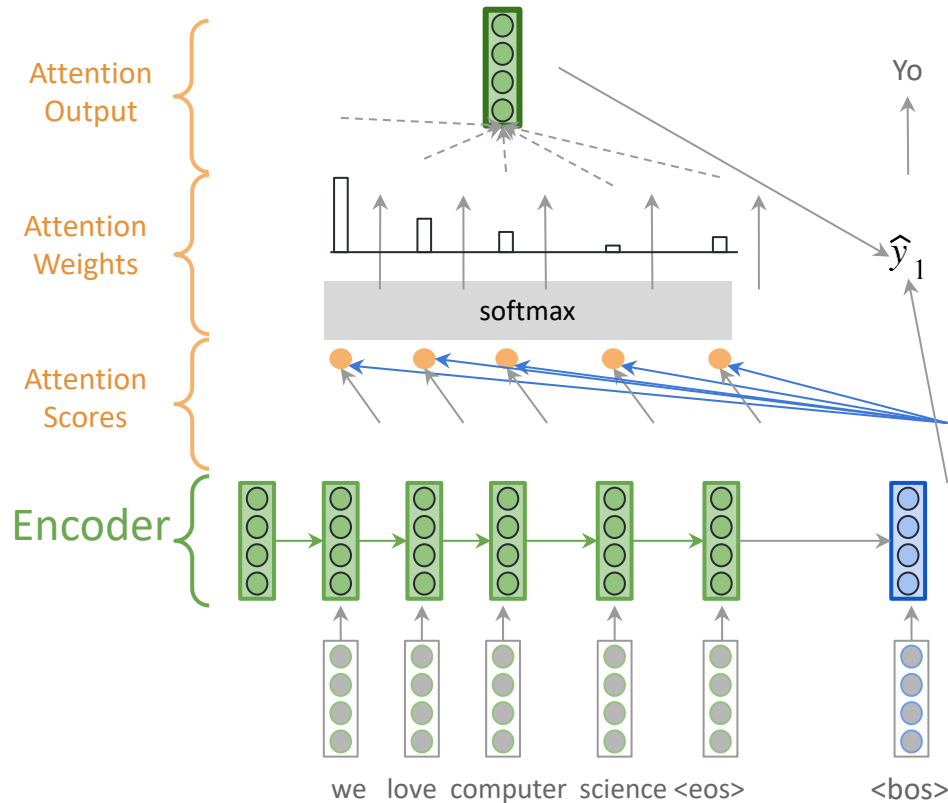
Used to make sure the value of qk^T does not become too large



ATTENTION

VS

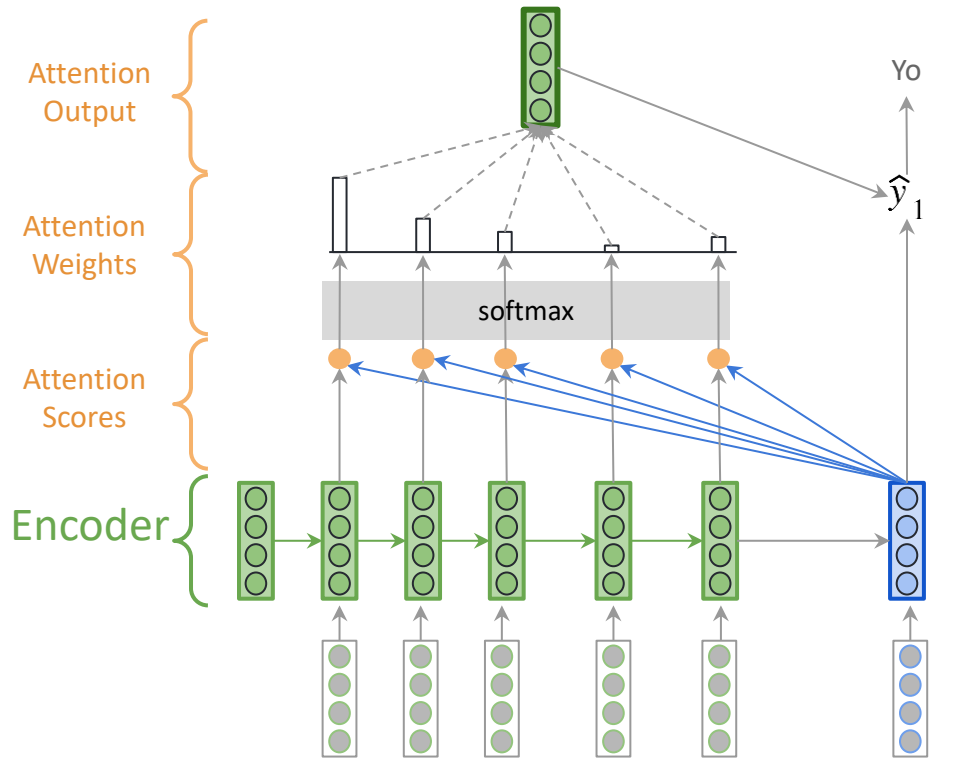
SELF-ATTENTION



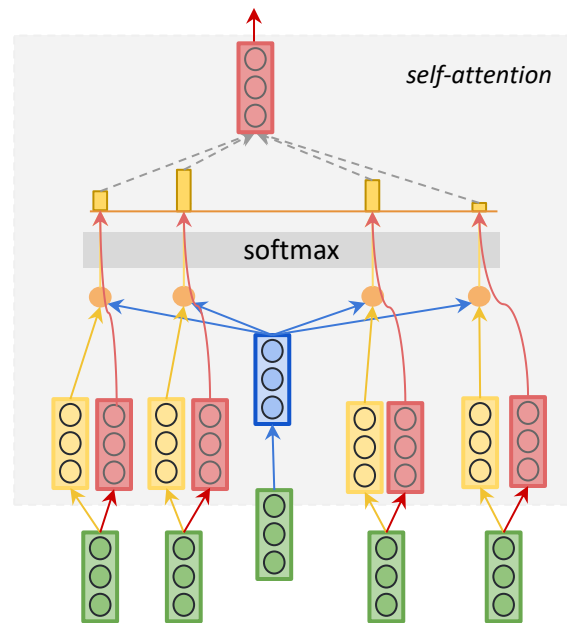
ATTENTION

VS

SELF-ATTENTION



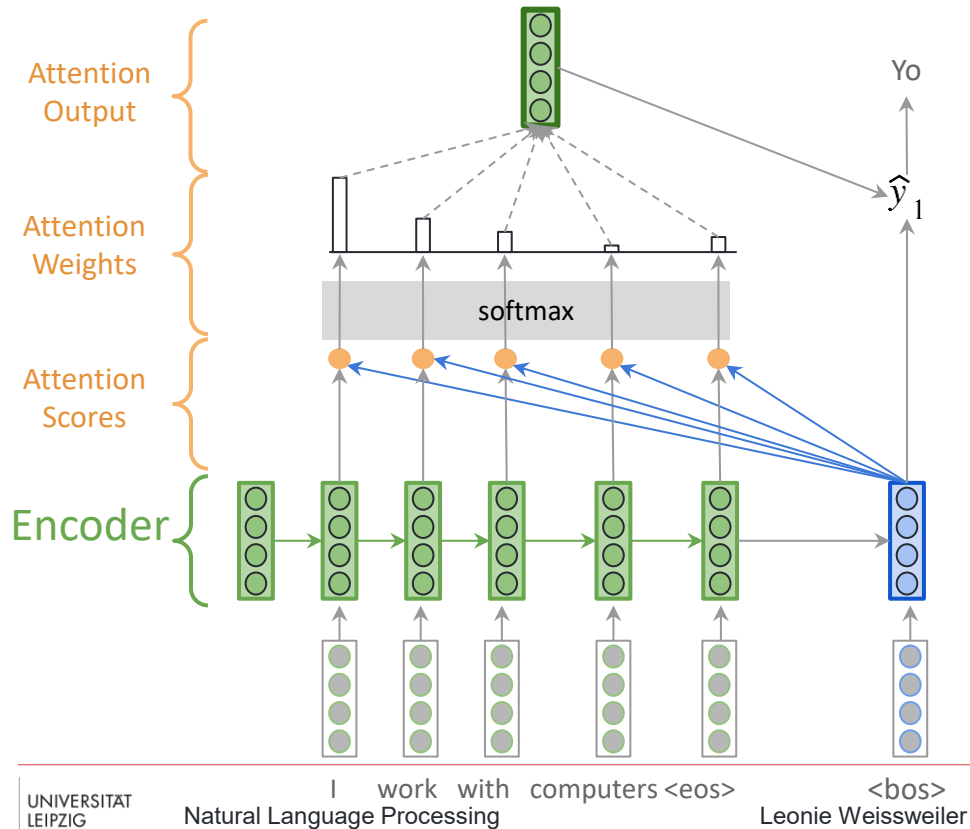
The query is similar to the current decoder step



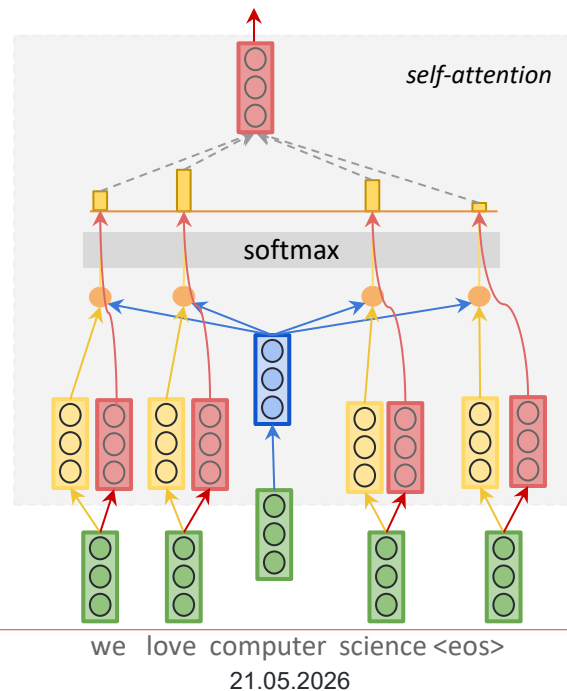
ATTENTION

VS

SELF-ATTENTION



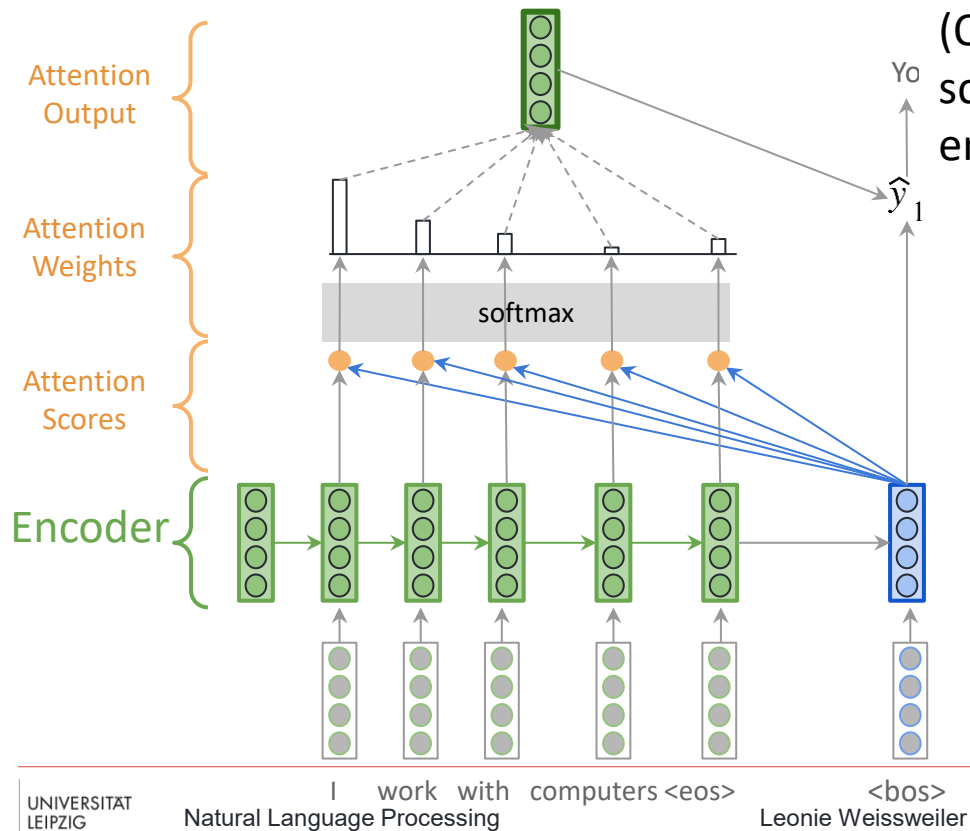
What about the Key?



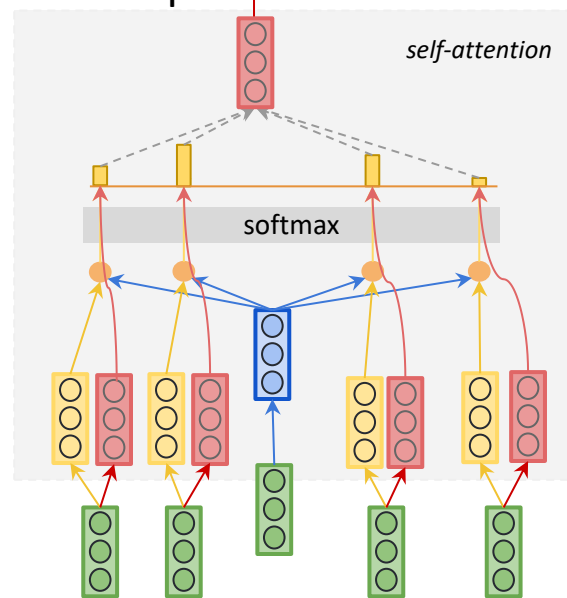
ATTENTION

VS

SELF-ATTENTION



(Query * Key) is similar to dot product scoring (where the key is equivalent to the encoder outputs)



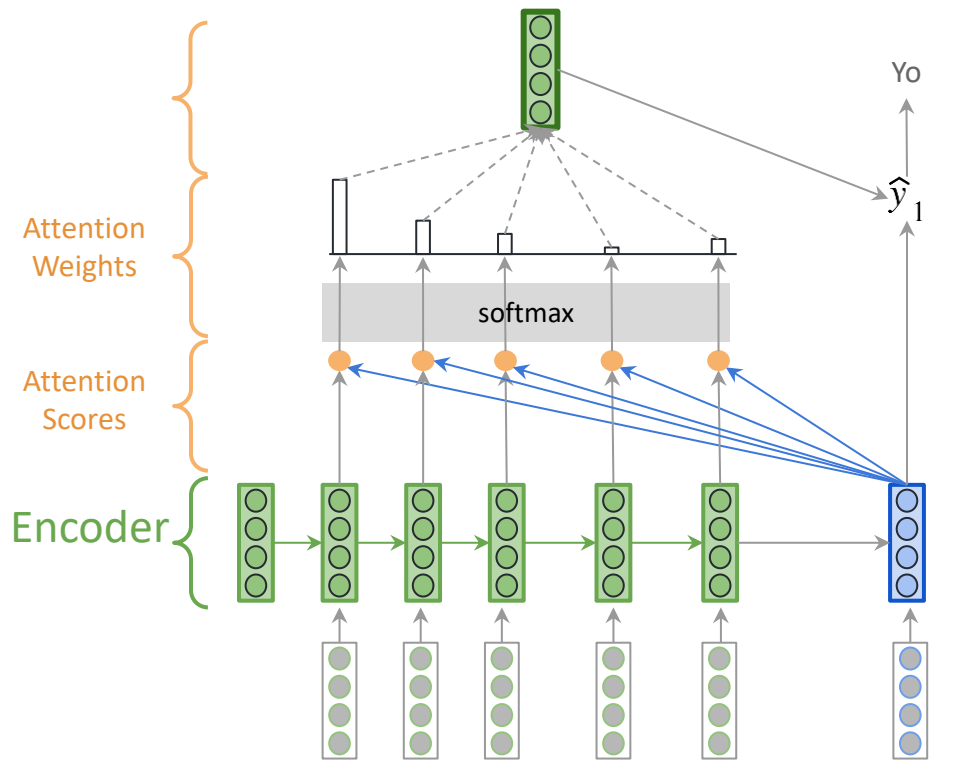
we love computer science <eos>

21.05.2026

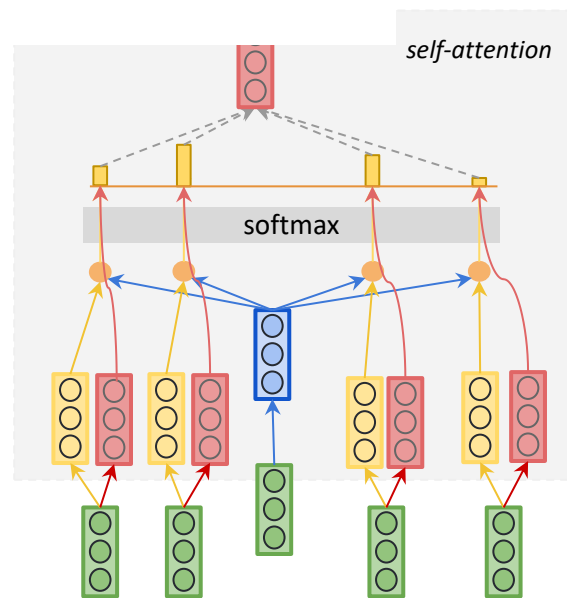
ATTENTION

VS

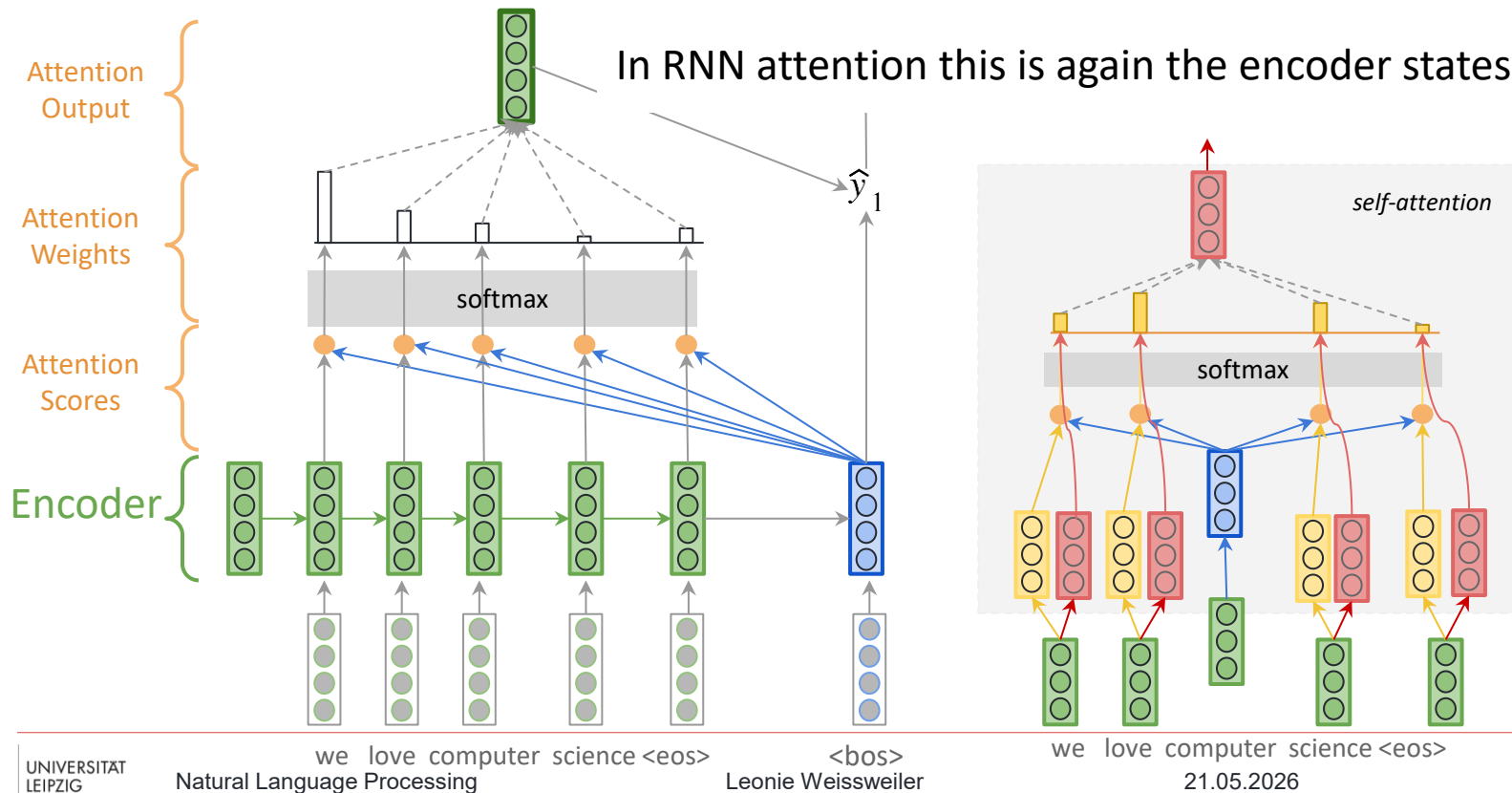
SELF-ATTENTION



And the Values?



ATTENT After pushing both through a softmax we multiply the result with the **value** in self-attention before summing it

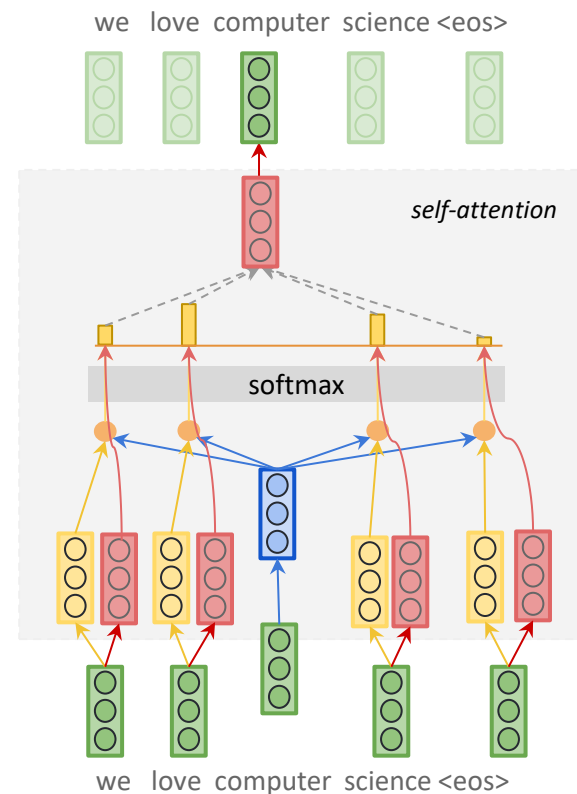


SELF-ATTENTION: QUERY, KEY, VALUE

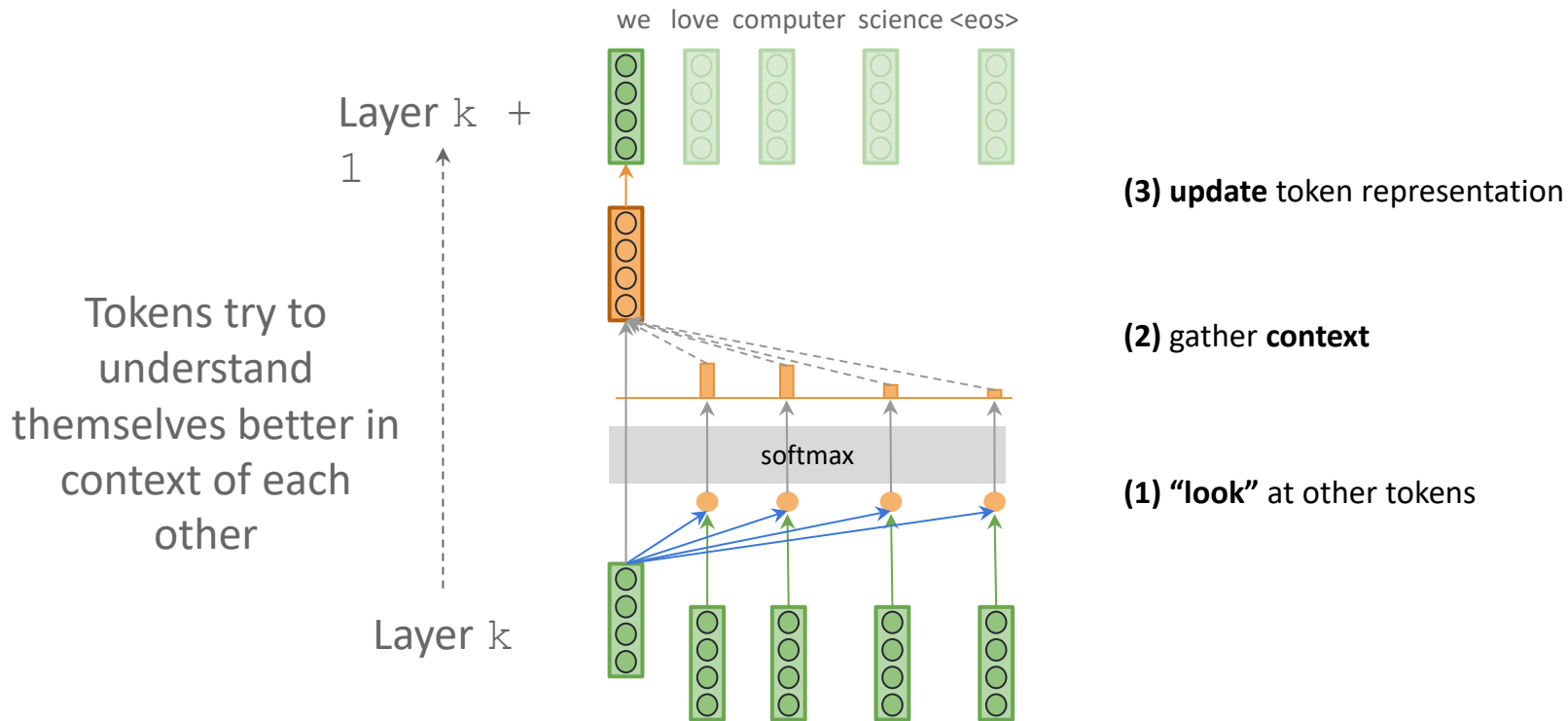
Each vector receives a:

- ⌘ **Query (q):** vector **from** which the attention is looking
- ⌘ **Key (k):** vector **at** which the query looks to establish context
- ⌘ **Value (v):** value of word being looked **at**, weighted based on context

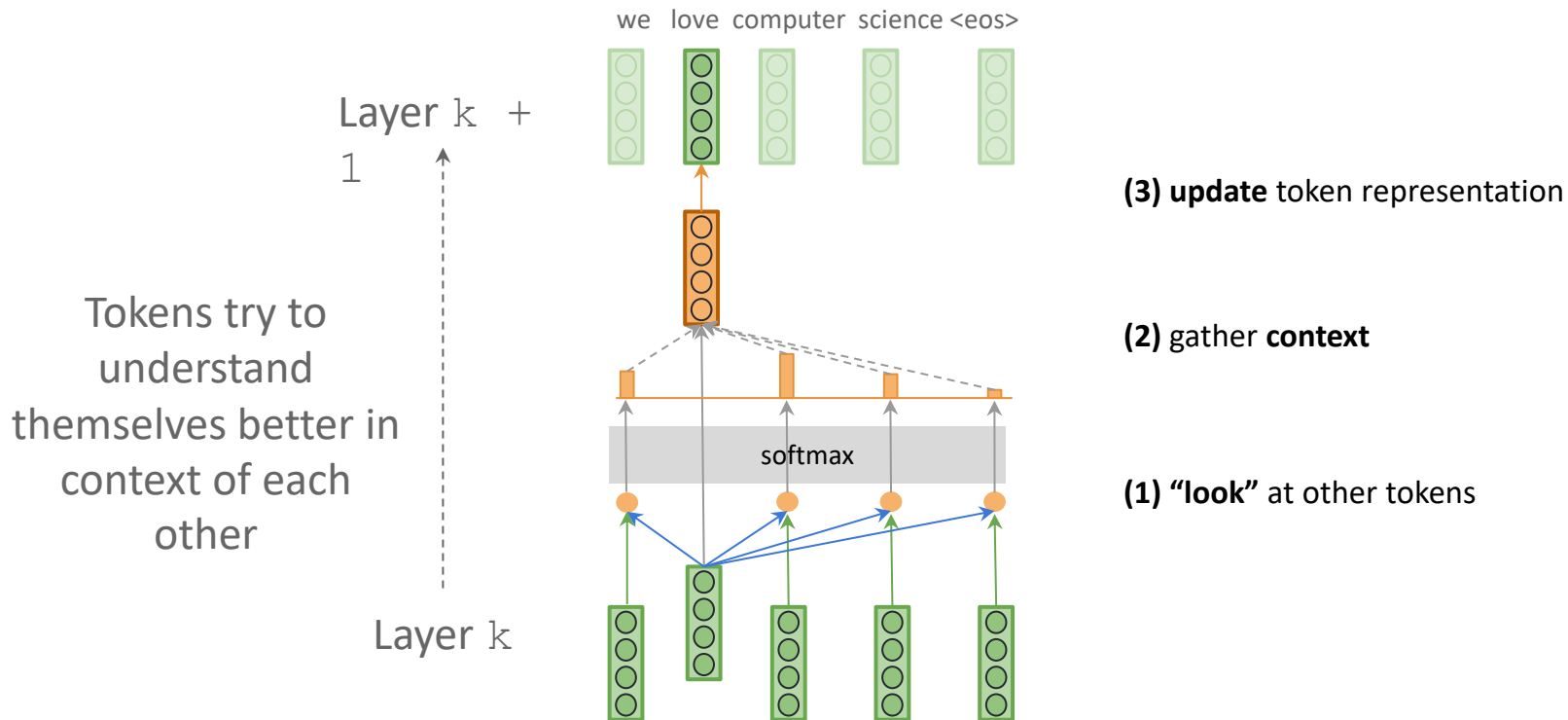
This is then fed to the next layer



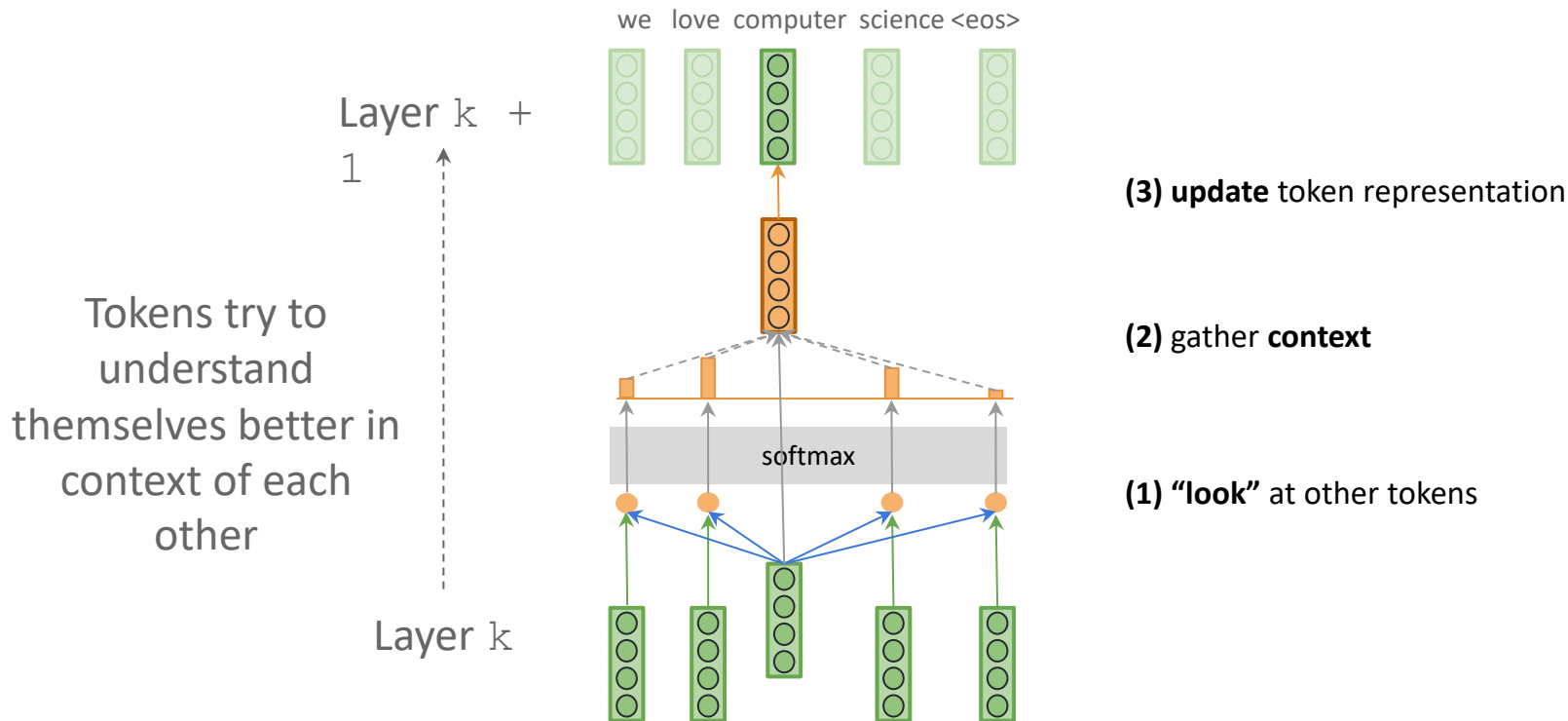
SELF-ATTENTION



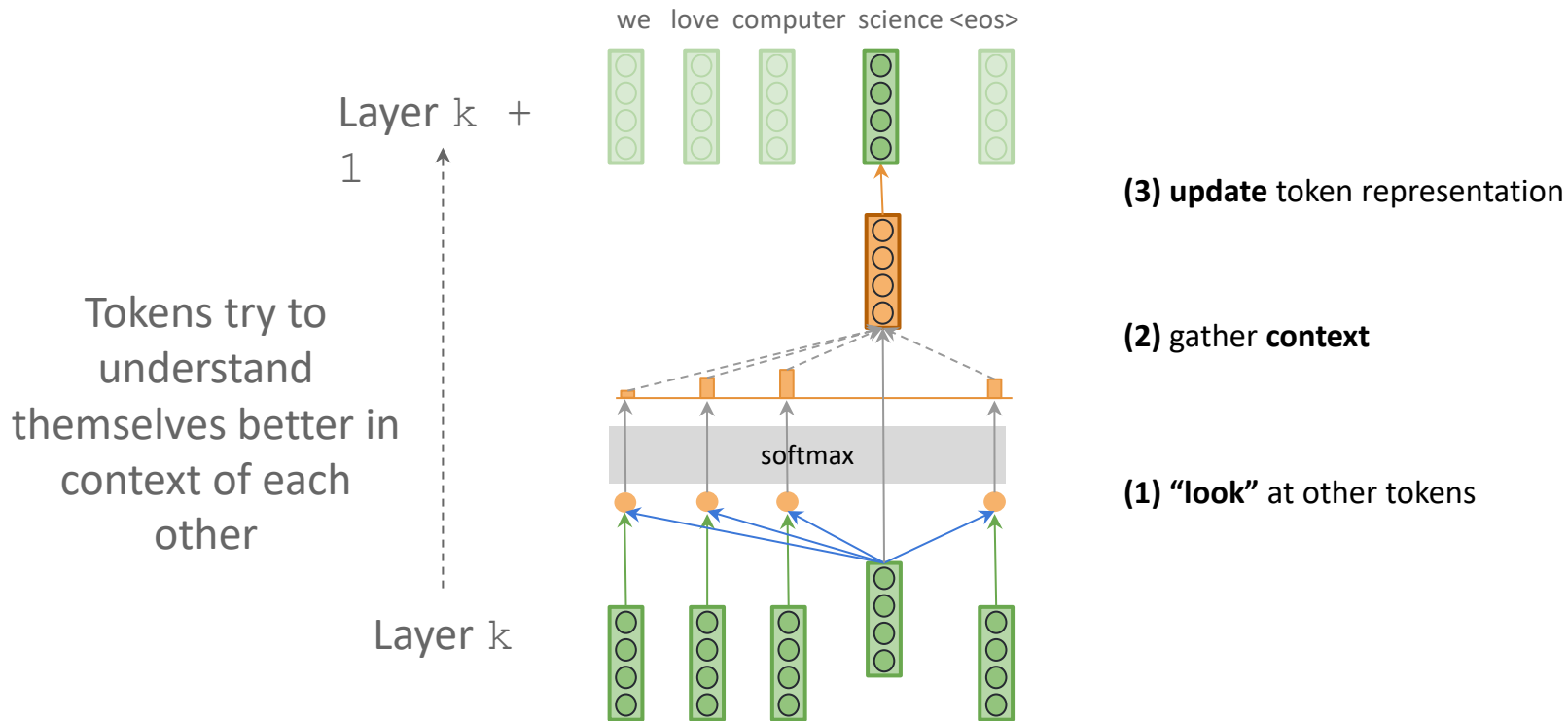
SELF-ATTENTION



SELF-ATTENTION



SELF-ATTENTION

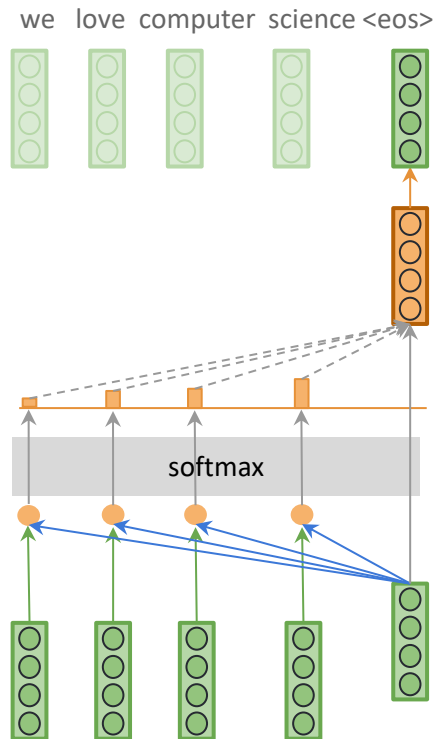


SELF-ATTENTION

Layer $k + 1$

Tokens try to understand themselves better in context of each other

Layer k



In practice, this happens **in parallel**

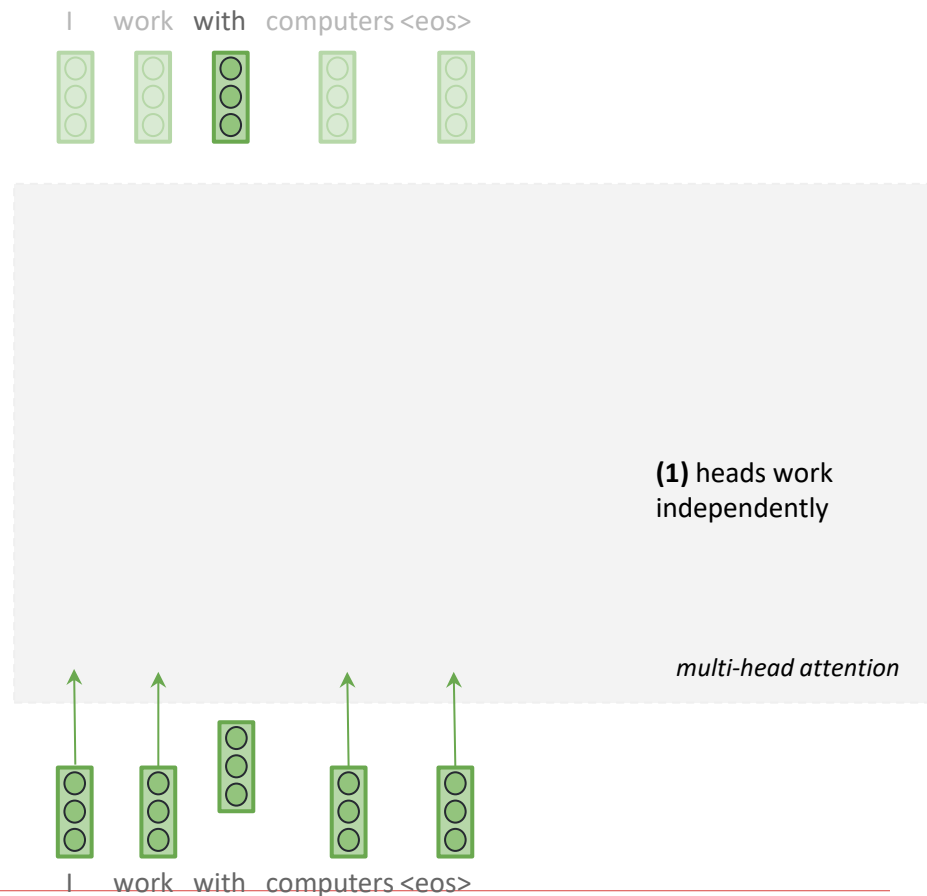
(3) **update** token representation

(2) **gather context**

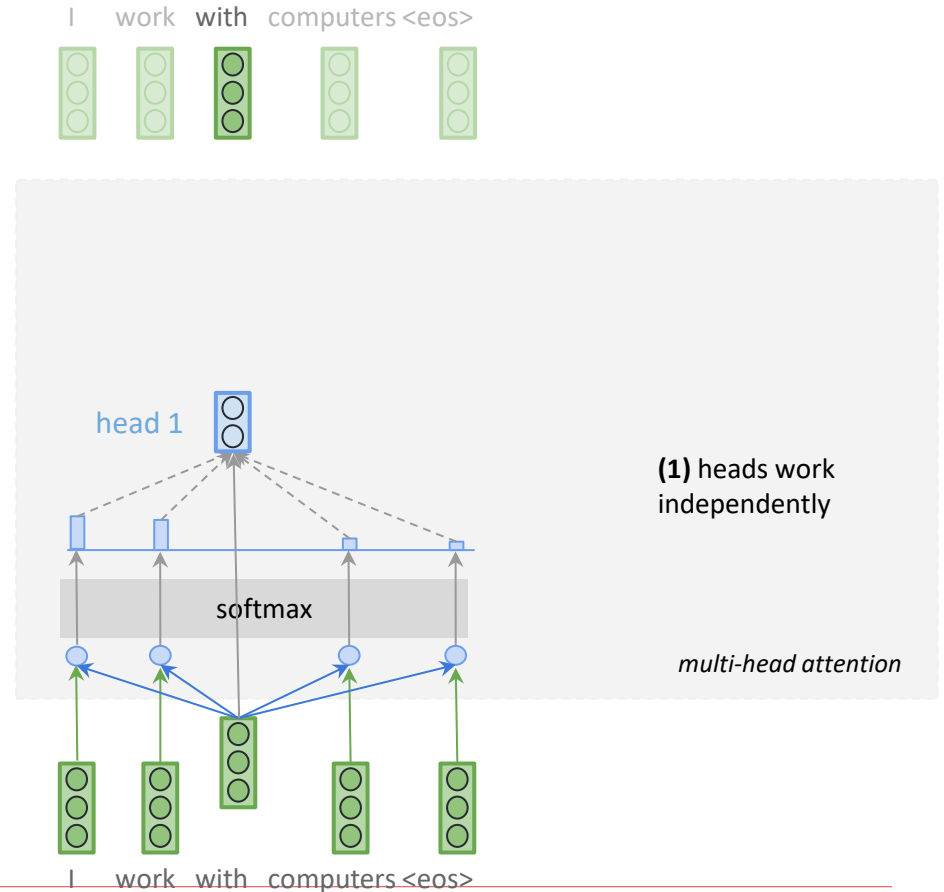
(1) **“look”** at other tokens

QUESTIONS?

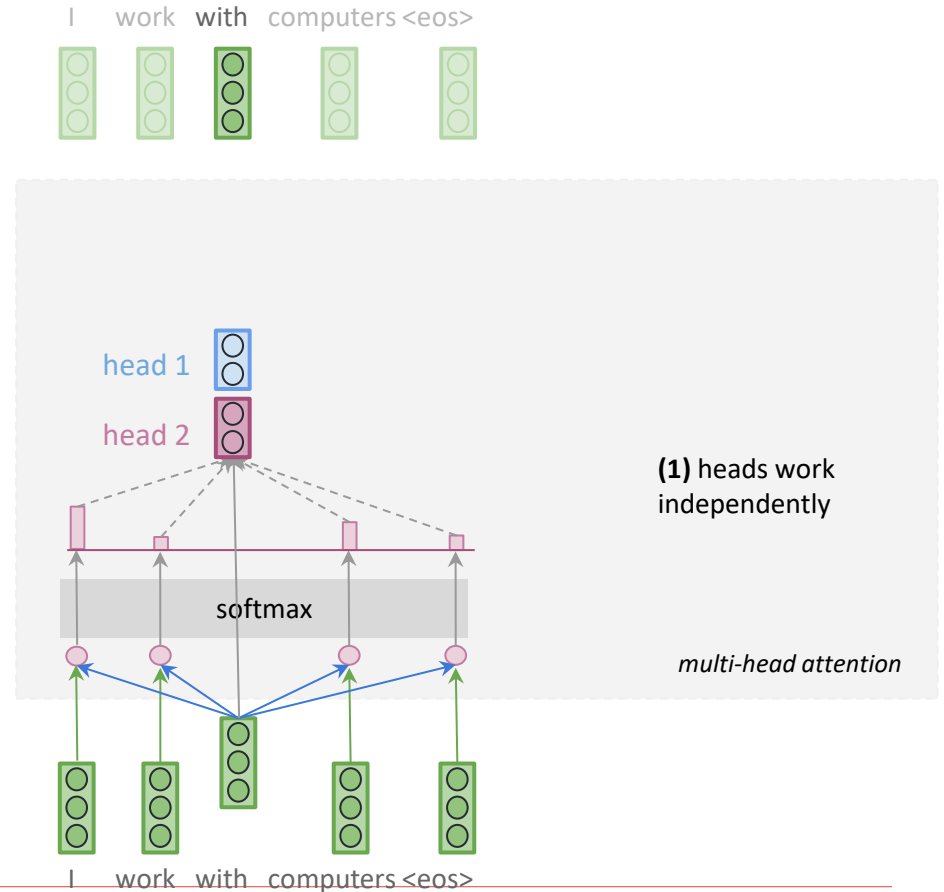
MULTI-HEAD ATTENTION



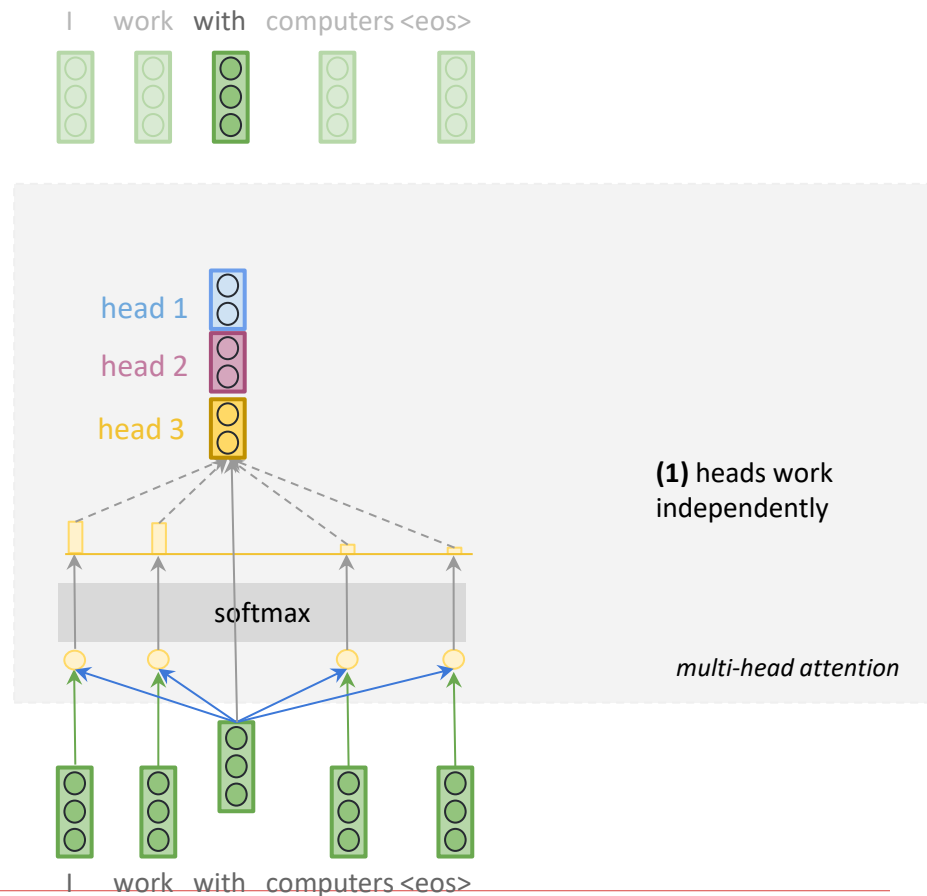
MULTI-HEAD ATTENTION



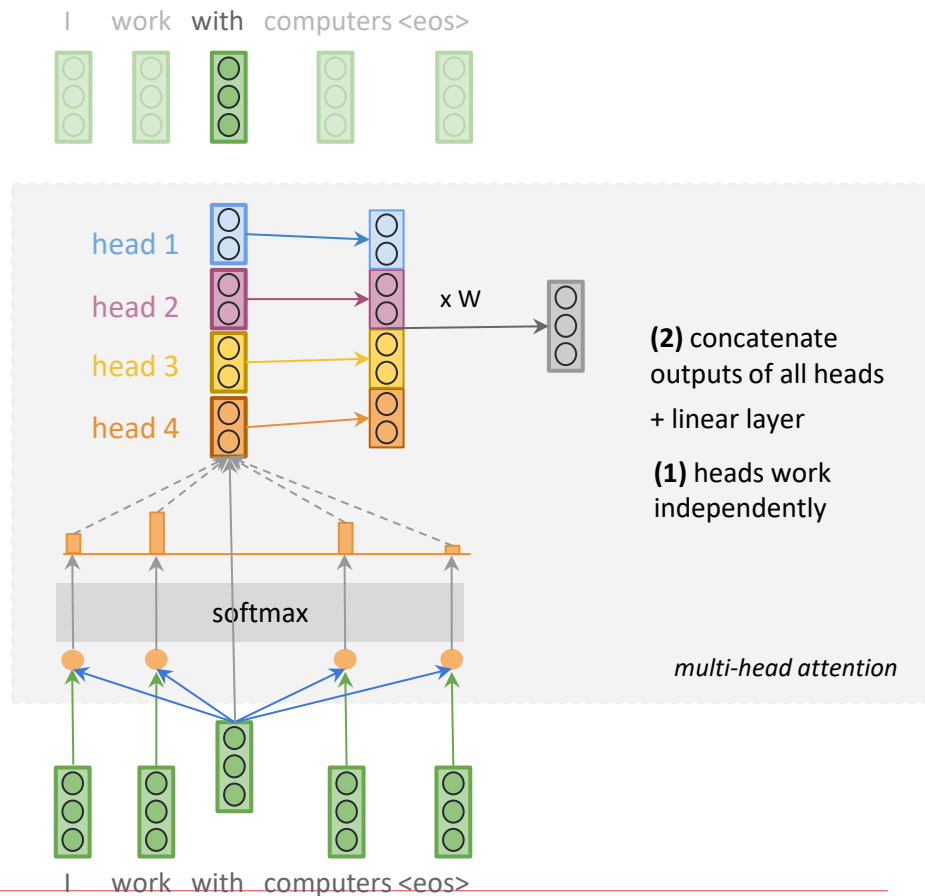
MULTI-HEAD ATTENTION



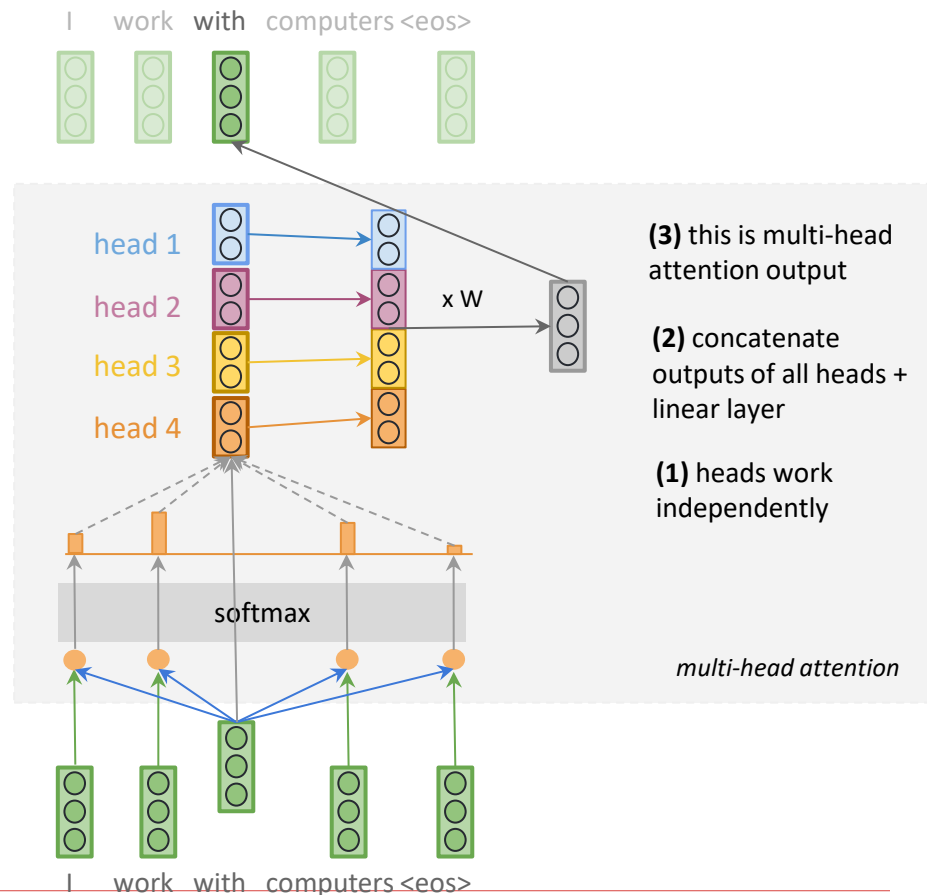
MULTI-HEAD ATTENTION



MULTI-HEAD ATTENTION



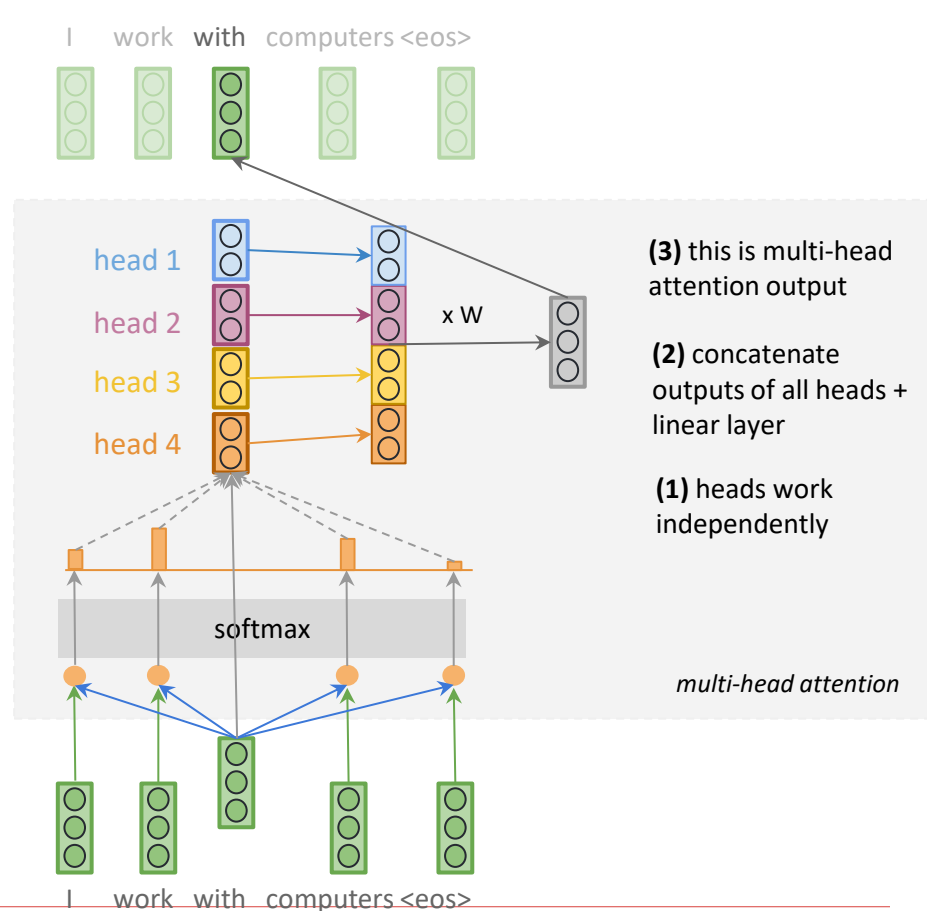
MULTI-HEAD ATTENTION



MULTI-HEAD ATTENTION

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

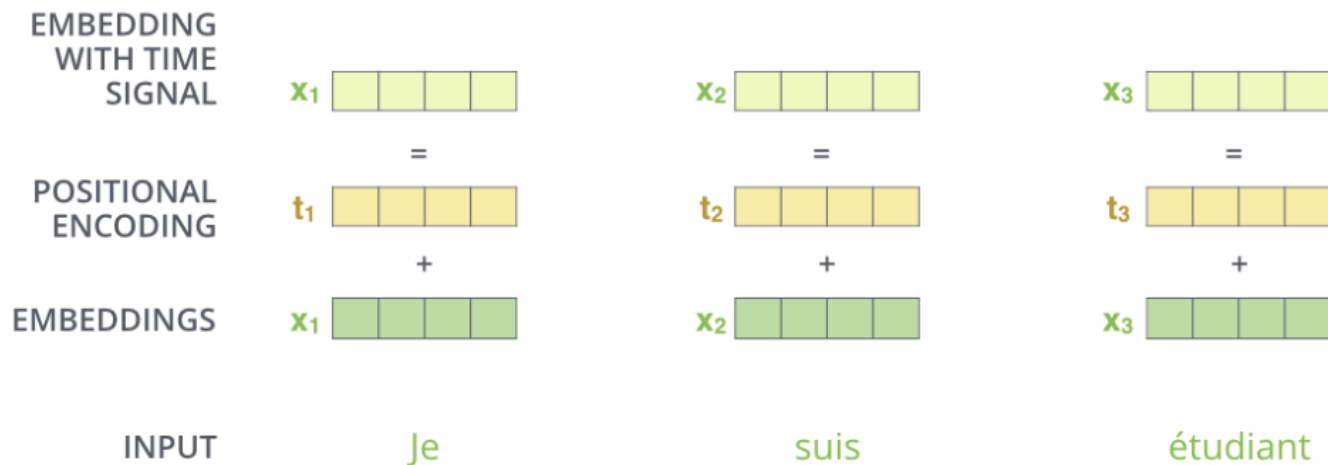
where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



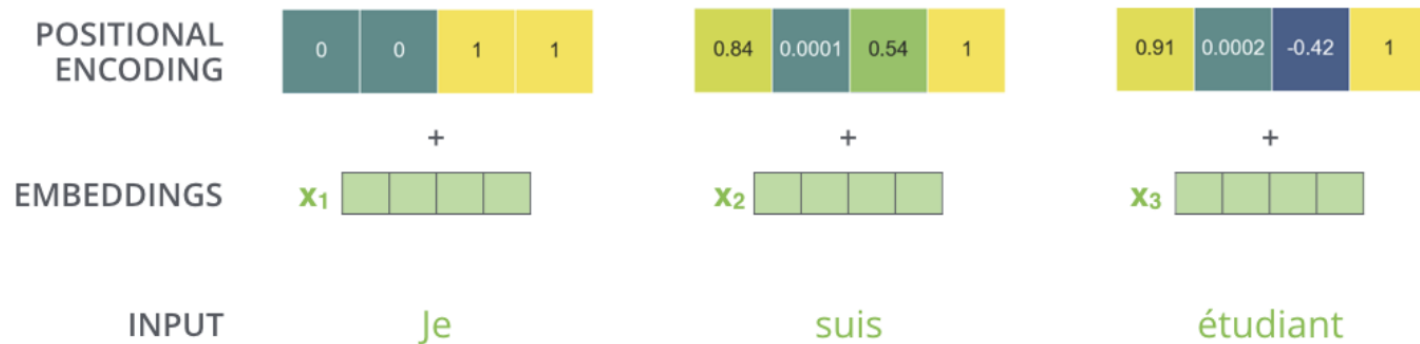
QUESTIONS?

MORE TRANSFORMER COMPONENTS

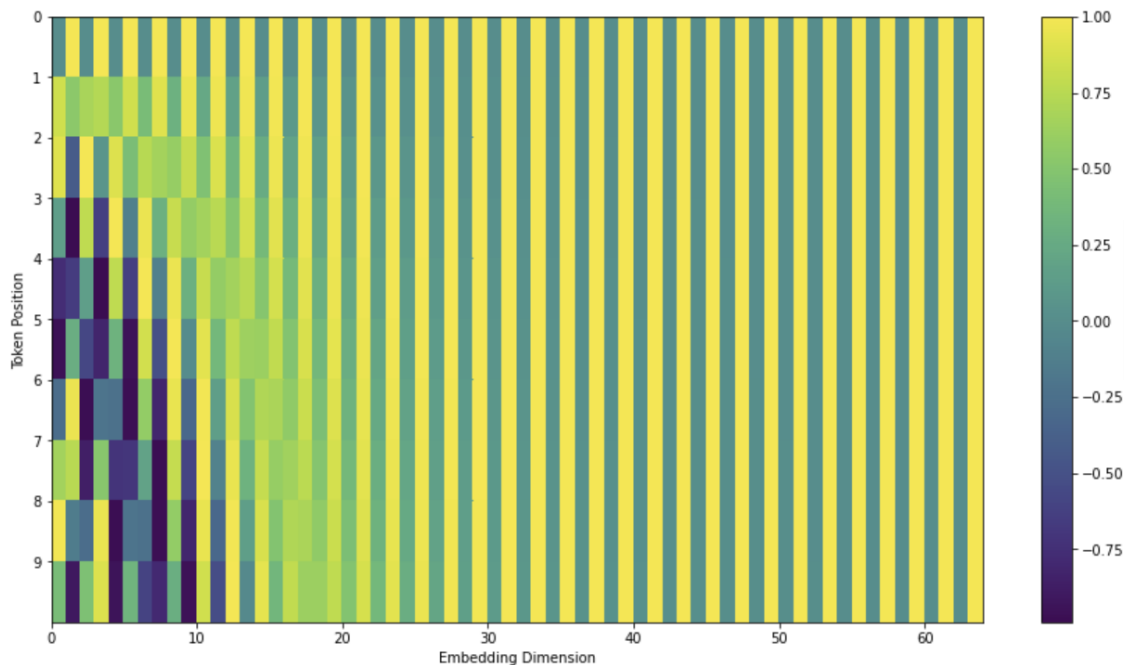
ADDING POSITION ENCODINGS TO EMBEDDINGS



REAL POSITIONAL ENCODINGS



GENERATING POSITIONAL ENCODINGS



$$\text{PE}_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

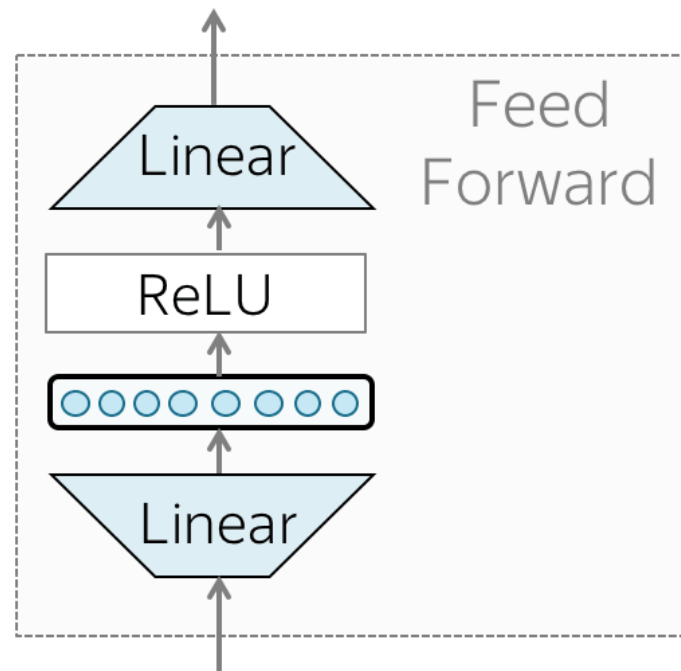
$$\text{PE}_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}}),$$

FEED-FORWARD BLOCKS

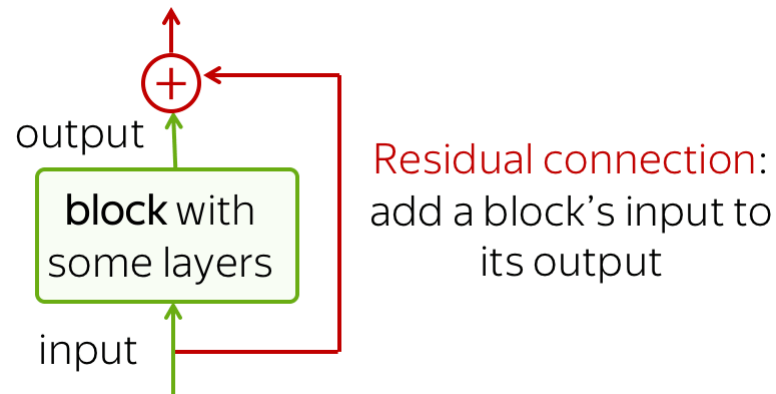
- In addition to attention, each layer has a feed-forward network block: two linear layers with ReLU non-linearity between them:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

- After looking at other tokens via an attention mechanism, a model uses an FFN block to process this new information (attention - "look at other tokens and gather information", FFN - "take a moment to think and process this information").



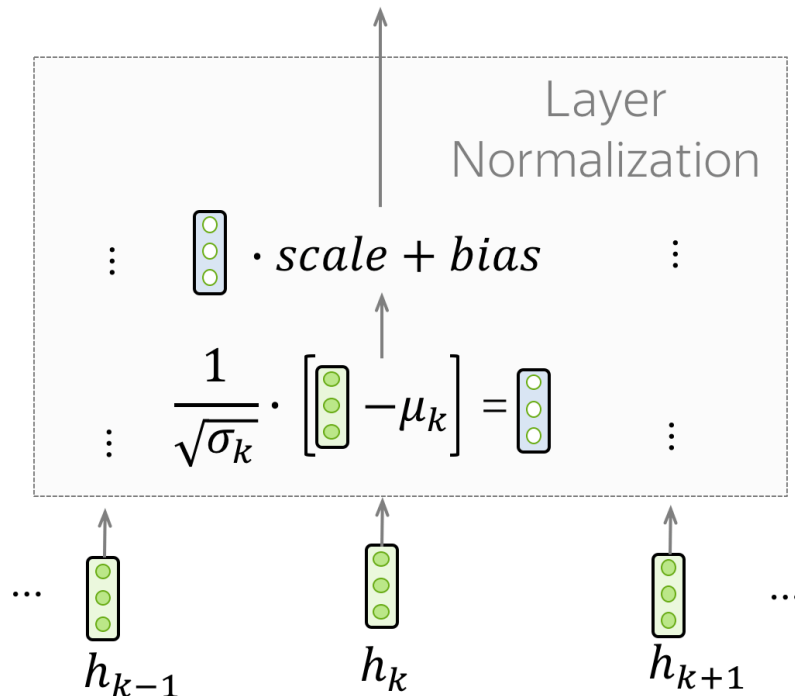
RESIDUAL CONNECTIONS



- Residual connections are very simple (add a block's input to its output), but at the same time are very useful: they ease the gradient flow through a network and allow stacking a lot of layers.
- In the Transformer, residual connections are used after each attention and FFN block. On the illustration above, residuals are shown as arrows coming around a block to the green "Add & Norm" layer. In the "Add & Norm" part, the "Add" part stands for the residual connection.

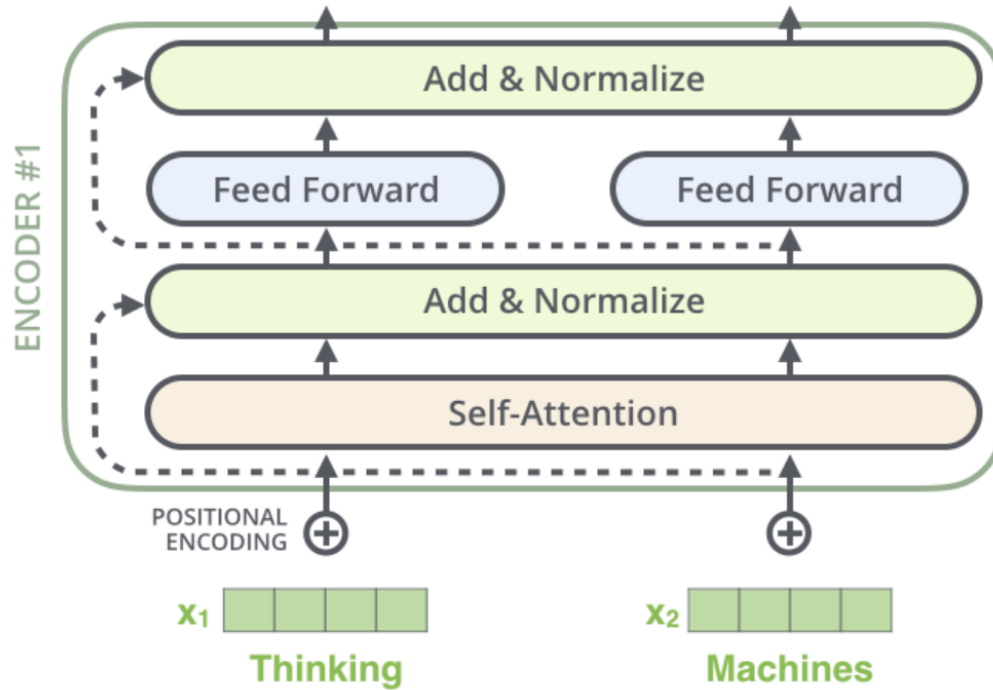
LAYER NORMALISATION

- independently normalizes vector representation of each example in batch - this is done to control "flow" to the next layer
- improves convergence stability and sometimes even quality
- LayerNorm has trainable parameters, scale and bias, which are used after normalization to rescale layer's outputs (or the next layer's inputs)
- Scale and bias are trainable, but they are layer parameters → the same for every example

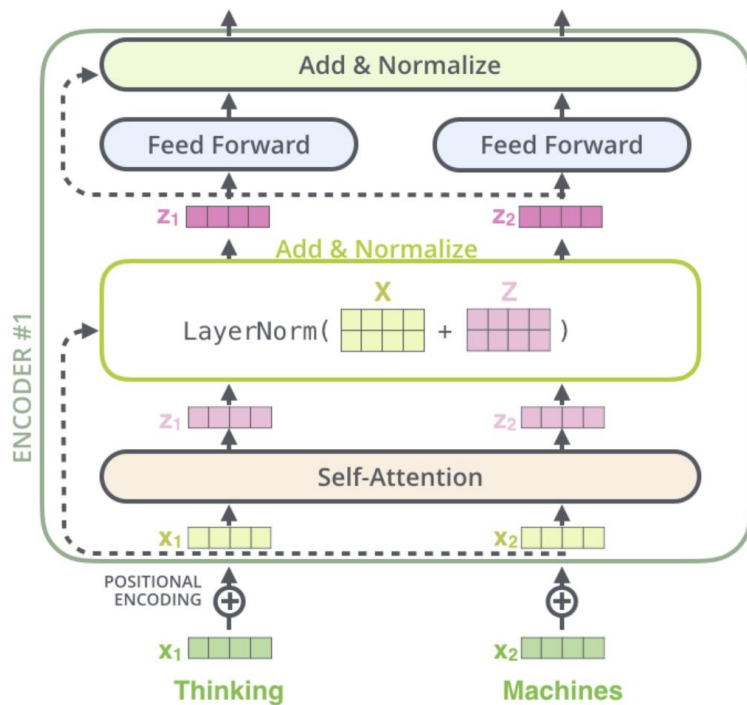


QUESTIONS?

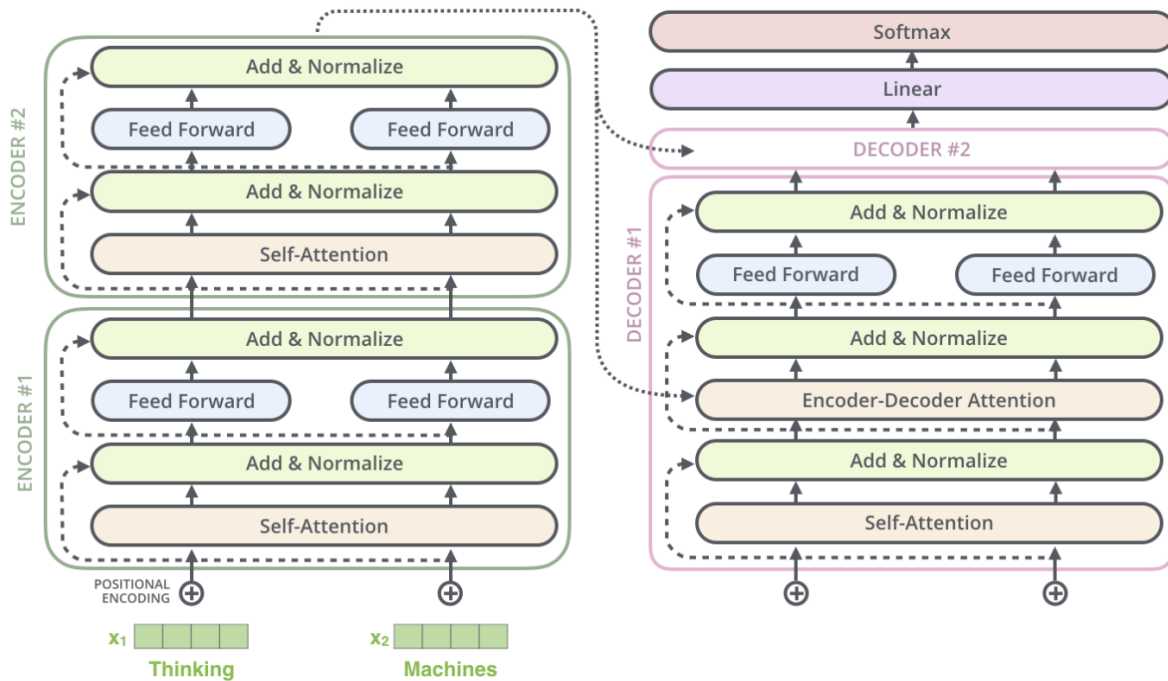
PUTTING IT ALL TOGETHER



RESIDUAL CONNECTION AND LAYER NORM

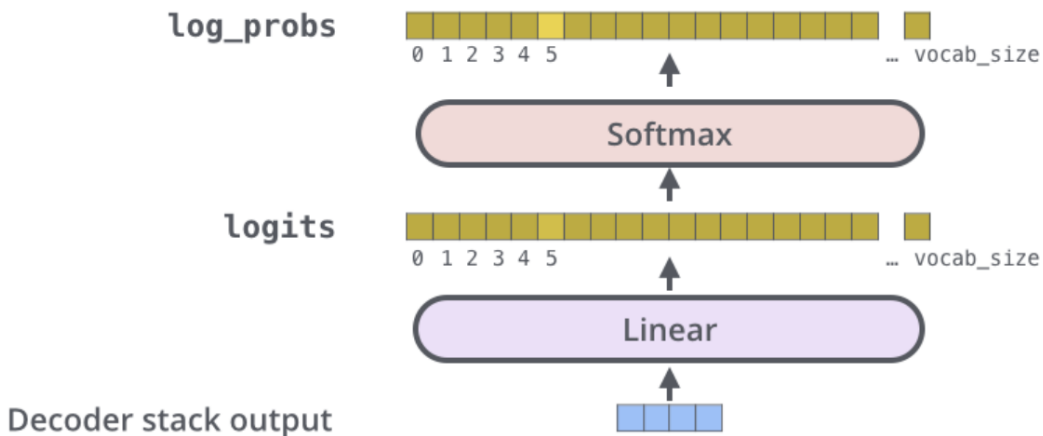


SMALL TRANSFORMER



Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(**argmax**)

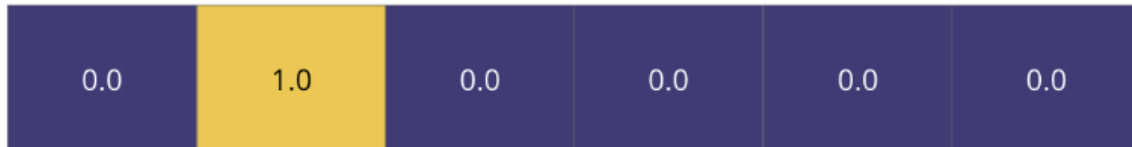


TRAINING A TRANSFORMER

Output Vocabulary

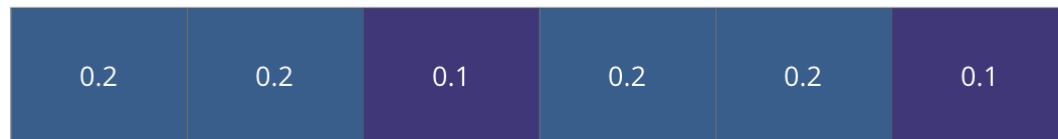
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"



THE LOSS FUNCTION

Untrained Model Output



Correct and desired output



a

am

I

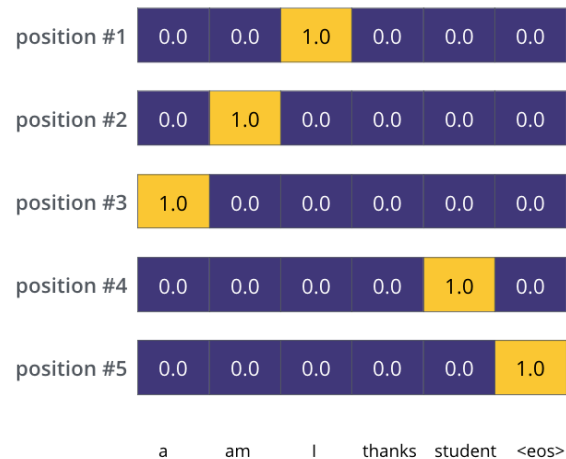
thanks

student

<eos>

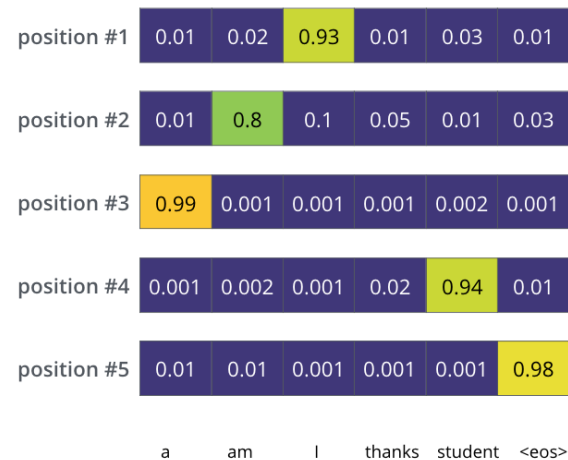
Target Model Outputs

Output Vocabulary: a am I thanks student <eos>

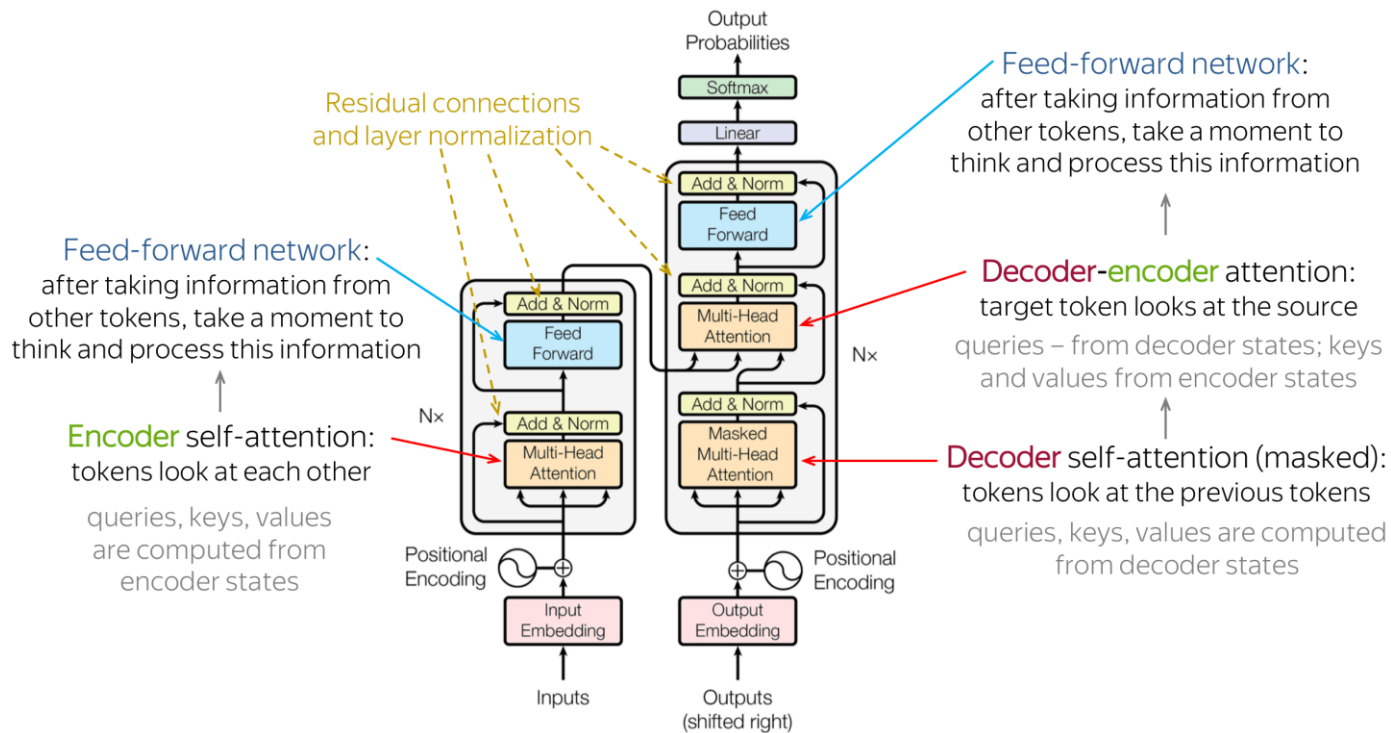


Trained Model Outputs

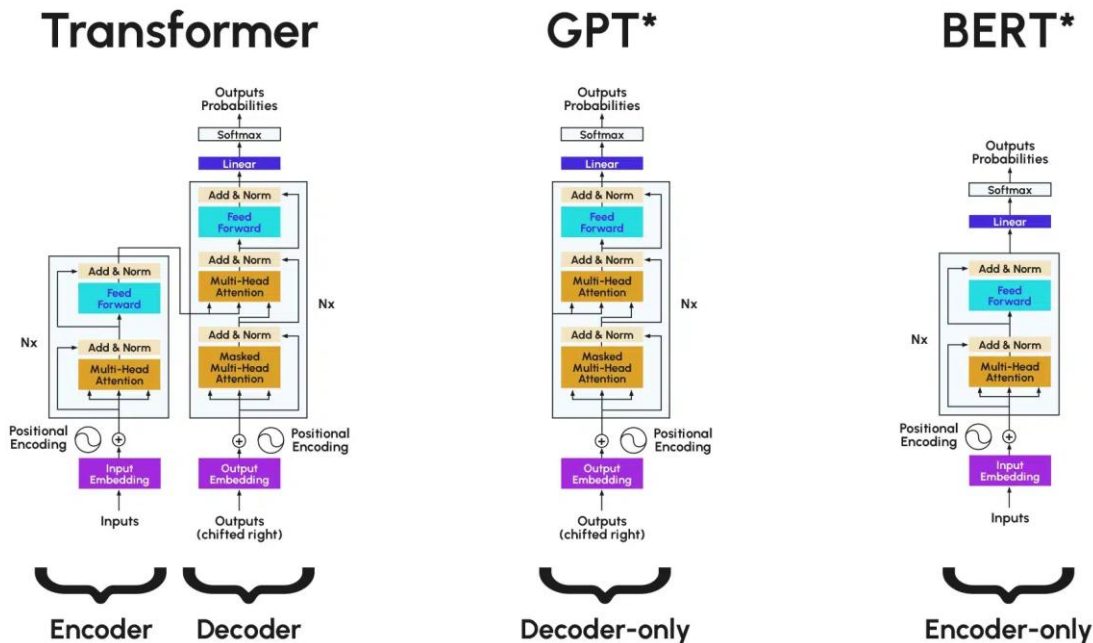
Output Vocabulary: a am I thanks student <eos>



PUTTING IT ALL TOGETHER



PREVIEW: ENCODER-ONLY AND DECODER-ONLY



*Illustrative example, exact model architecture may vary slightly

QUESTIONS?